simpleja4p4 — detailed architecture

JA4+ fingerprinting in a P4 switch: every block, every flow, every code file, and why it is built this way

Kept in step with the code: every change and fix is recorded here

Version of October 12, 2026

blue	P4 program in the switch (data plane)
green	controller (Python, P4Runtime)
orange	hosts, network, external programs
purple	references and test tools (independent of the project)
grey	files: inputs, outputs, results

Contents

I Architecture **4**

1 The system at a glance **4**

1.1 Who does what, and why the line is drawn there . . . 5

1.2 Map of the code: every file, what it feeds . . . 6

1.3 Build and run: what each command produces . . . 6

2 Inside the switch **7**

2.1 Life of a packet through the pipeline . . . 7

2.2 The parser: top-level state machine . . . 8

2.3 Ingress: the order of the controls . . . 10

2.4 Per-connection state . . . 11

3 Switch and controller: every channel between them **12**

3.1 The six digests . . . 12

3.2 A message split over segments, arriving out of order . . . 13

3.3 TLS 1.2 certificates (JA4X) and the SSH flush . . . 16

4 Each fingerprint, end to end **17**

4.1 JA4T, JA4TS, JA4L: the TCP handshake . . . 17

4.2 JA4 and JA4S: TLS hellos . . . 17

4.3 JA4H: HTTP requests . . . 17

4.4 JA4SSH: SSH packet sizes . . . 18

4.5 JA4D and JA4D6: DHCP . . . 18

4.6 JA4X: certificates . . . 18

5 How correctness is checked **18**
5.1 The replay bench (no root) . . . 18
5.2 Comparison against independent references . . . 19
5.3 The live run (root) . . . 19
5.4 The tests: what each one protects . . . 20
5.5 The measurement study . . . 20

II Explanation **21**

6 The problem and the scope **21**

7 Design decisions: what, why, and why not the alternatives **21**
7.1 D1. BMv2, v1model and a P4Runtime controller . . . 22
7.2 D2. The parser reads every fingerprint value . . . 22
7.3 D3. Results leave the switch as digests . . . 22
7.4 D4. Sorting, SHA-256 and text in the controller . . . 22
7.5 D5. Walked packets are resubmitted and forwarded on a second pass . . . 23
7.6 D6. Split and out-of-order messages are joined by the controller and read by the switch . . . 23
7.7 D7. Copies are made at egress (E2E clone) . . . 23
7.8 D8. Per-connection state in hashed register slots . . . 23
7.9 D9. One `digest()` call per control; the handshake digest sent last . . . 24
7.10 D10. A serial number in DHCP, HTTP and SSH digests . . . 24
7.11 D11. GREASE is recognised in the parser's `select` . . . 24
7.12 D12. Capacities: 128 / 64 / 64 values, 512 / 1,024 / 256 bytes, then "incomplete" 24
7.13 D13. JA4L timed in the switch . . . 24
7.14 D14. SSH: counts in the switch, the mode in the controller . . . 25
7.15 D15. The last SSH window through a flush packet . . . 25
7.16 D16. JA4X: the controller cuts certificates out, the switch walks DER . . . 25
7.17 D17. Byte-per-state HTTP walk with chained 2,048-bit fields . . . 25
7.18 D18. A rootless replay bench next to the live network . . . 25
7.19 D19. FoxIO's own tools as the reference . . . 26
7.20 D20. One design only . . . 26
7.21 D21. A number for every connection (port reuse) . . . 26
7.22 D22. Rule (e): pieces ahead of a gap in an HTTP direction . . . 26
7.23 D23. VLAN tags and SSH on any port . . . 27
7.24 D24. Bounded controller buffers . . . 27

8 Every code artifact **27**
8.1 Data plane (P4) . . . 27
8.2 Controller (Python) . . . 30

8.3 Tools, tests, network . . . 31

9 The algorithms, step by step **32**

9.1 Connection key (`main.p4`, ConnKey) . . . 32

9.2 Connection number (`main.p4`, ConnId) . . . 33

9.3 TLS hello walk (`parser_tls.p4`) . . . 33

9.4 HTTP request walk (`parser_http.p4`) . . . 33

9.5 DER walk with return codes (`parser_x509.p4`) . . . 34

9.6 Handshake and JA4L (`handshake.p4`) . . . 34

9.7 Which segments are copied (`stream.p4`) . . . 34

9.8 Joining (`reassembly.py`) . . . 34

9.9 JA4SSH (`ssh.p4`, `ssh.py`) . . . 35

9.10 Text and hashes (controller) . . . 35

9.11 Verdicts (`compare.py`) . . . 35

10 Corner cases and how each is handled **35**

11 Problems found and how they were fixed **38**

12 Results **42**

12.1 FoxIO’s sample captures, replayed . . . 42

12.2 The live run . . . 43

12.3 Capacities: the measurement study . . . 43

12.4 Tests . . . 43

13 What is done, what is not, and why **44**

14 Assumptions, and what changes on a real network **46**

15 Known limits **48**

16 Questions and answers **49**

Part I

Architecture

1 The system at a glance

What it does. Two hosts talk through one P4 switch. Before any protocol encrypts its session, the two sides exchange clear-text messages (TCP handshake, TLS hellos, HTTP request headers, SSH packet sizes, DHCP options, TLS 1.2 certificates). The switch reads those messages in its P4 parser and pipeline while forwarding every packet unchanged, and produces the ten JA4+ fingerprints. A controller next to it writes the final text.

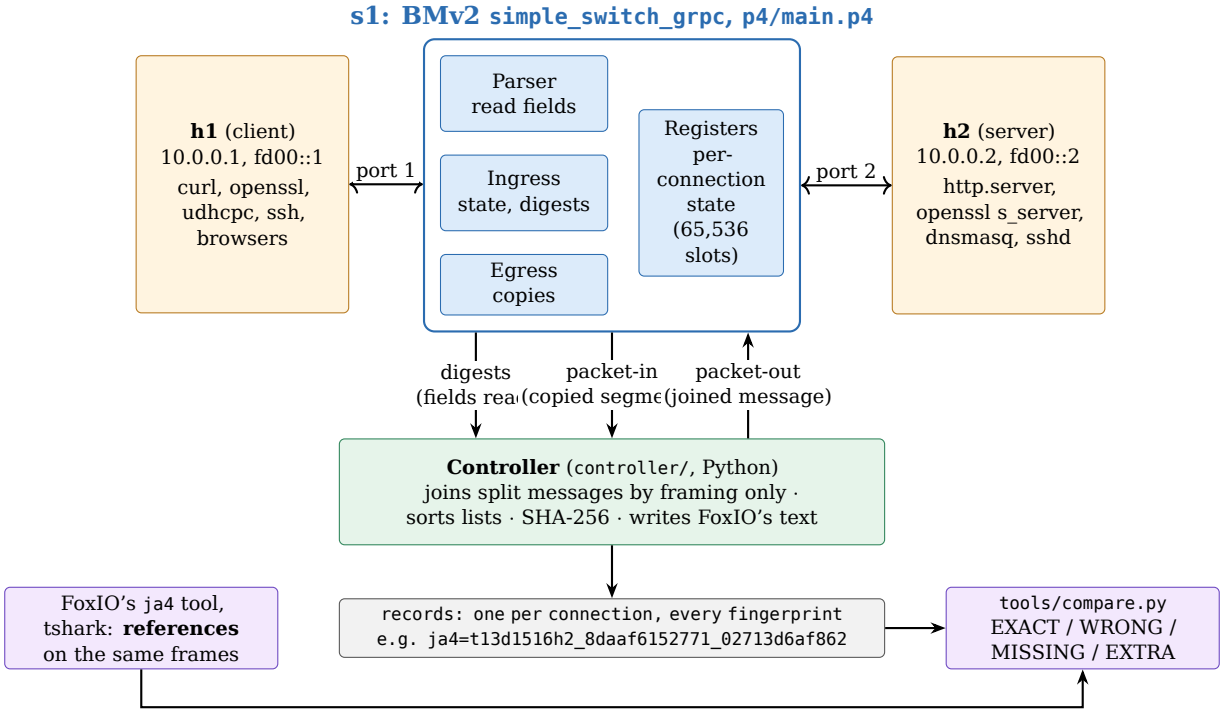


Figure 1: The whole system. Every packet is forwarded unchanged between the hosts; the switch reads the clear-text exchange and reports what it read; the controller only orders bytes and writes text; independent tools check the result.

1.1 Who does what, and why the line is drawn there

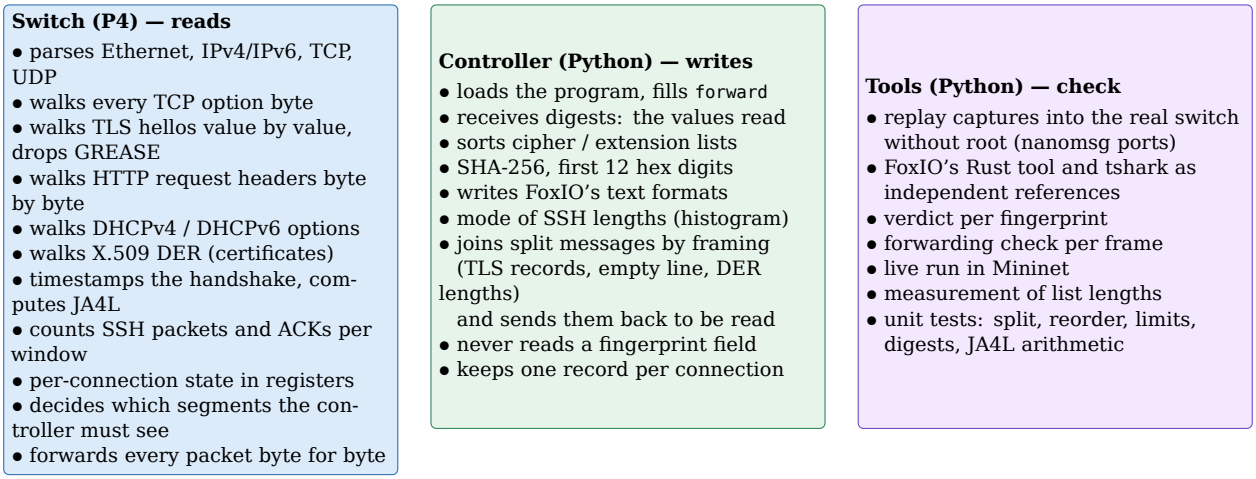


Figure 2: The division of work. The line: everything that must look at packet bytes to extract a fingerprint value is in P4; what P4 on this target cannot do (sort, SHA-256, a histogram, text) is in the controller, working only on values the switch extracted.

Why: The project's claim is "fingerprinting done in P4". Reading the clear-text exchange (the variable-length parsing, GREASE, completeness, per-connection state, timing) is the hard part and is done in the data plane. The v1model / BMv2 target has no SHA-256 extern, no loops outside the parser, no sorting, and no strings; those last steps are cheap, need no packet bytes, and are done by the controller on the switch's output.

Why not something else: (1) The controller could parse whole packets sent to it (packet-in of every packet): that would be a software fingerprinter with a switch in front, not P4 fingerprinting. (2) Sorting and hashing in P4: possible only with a sorting network sized to the list (many pipeline stages on hardware) and a hash the target does not have; the result would be identical and the claim no stronger. (3) No controller at all: the switch would have nowhere to put the text, and split messages could not be joined (a P4 switch cannot hold packets back).

1.2 Map of the code: every file, what it feeds

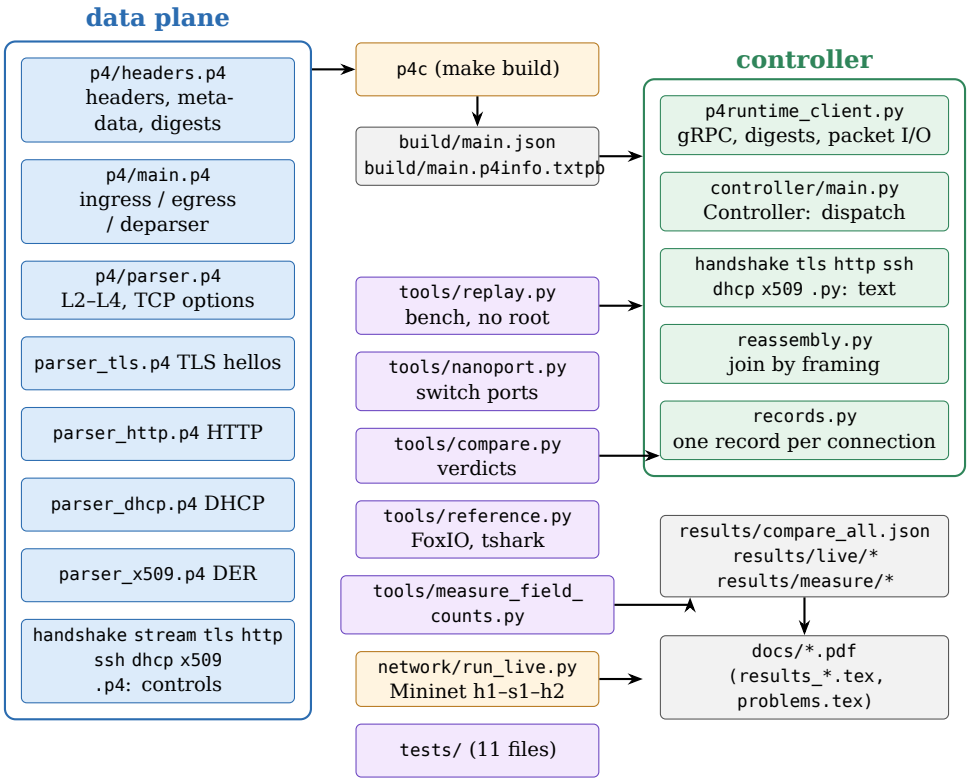


Figure 3: Every code file and what it feeds. `make build` compiles the P4 files into the program the switch runs; the controller loads it and talks to the switch; the tools drive switch + controller (replay without root, or the live Mininet network), compare with the references and write results, which the PDFs include.

1.3 Build and run: what each command produces

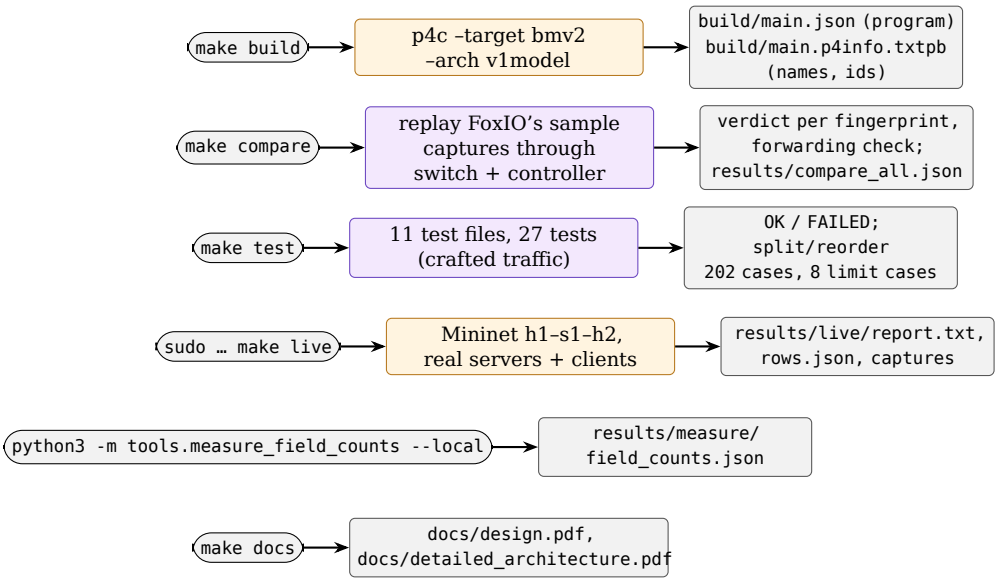


Figure 4: Commands and their outputs. Only `make live` needs root (Mininet's network namespaces).

2 Inside the switch

2.1 Life of a packet through the pipeline

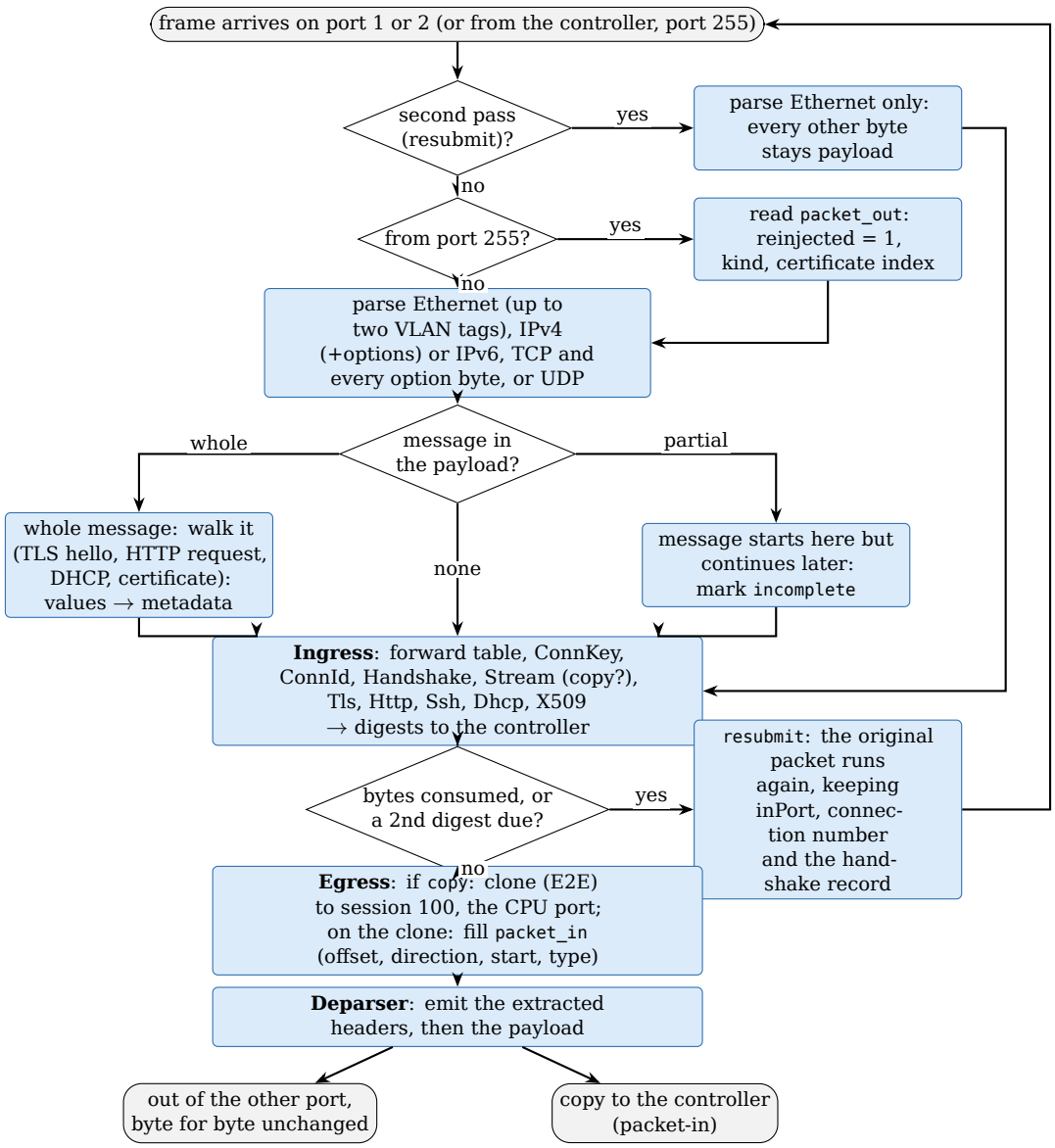


Figure 5: One packet's path (`p4/parser.p4`, `p4/main.p4`). A walk reads a message with `advance`, which removes the read bytes from the packet: such a packet is resubmitted, and on the second pass only Ethernet is parsed, so the original bytes are forwarded. A packet from the controller (port 255) is read the same way but never forwarded.

Why: `advance` is the only way for a v1model parser to skip bytes it does not extract; extracting every byte into headers would need header stacks thousands of entries long and the deparser would have to re-emit all of them. Skipping and forwarding the untouched original on a second pass keeps the walk cheap and the forwarding exact (checked frame by frame: 9,013 of 9,013).

Why not something else: Clone first, then walk the clone: an ingress clone is parsed again from the start, so it would lose its bytes the same way. Recirculate: goes through egress and the deparser, which would emit the already shortened packet.

2.2 The parser: top-level state machine

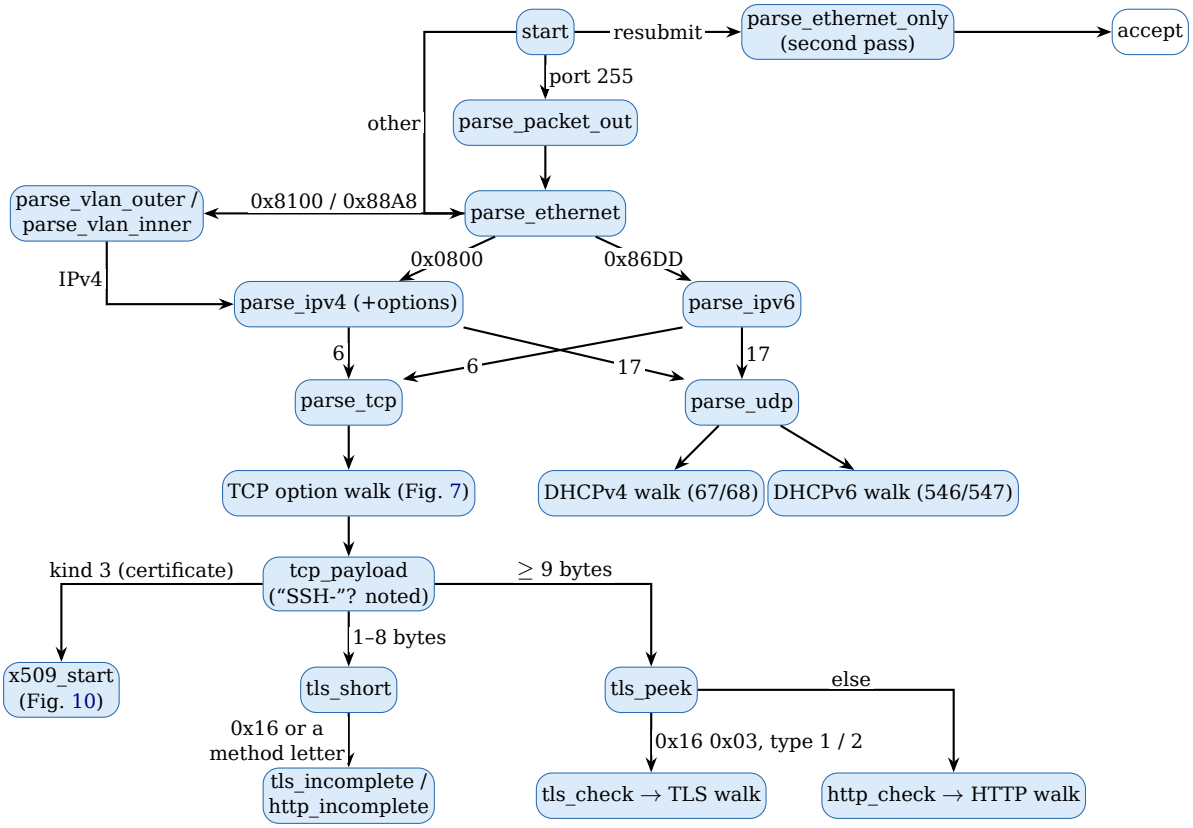


Figure 6: The parser's top level (`parser.p4` with `parser_tls.p4`, `parser_http.p4`, `parser_dhcp.p4`, `parser_x509.p4` included inside it). IPv4 fragments other than the first are not read beyond IP. Anything not matched goes to `accept`: the packet is still forwarded.

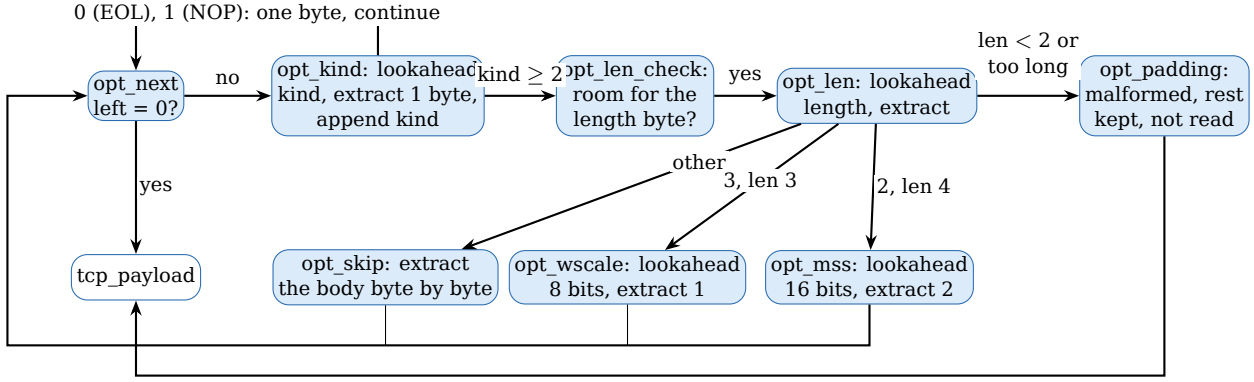


Figure 7: TCP option walk (`parser.p4`): `left` = data offset × 4 − 20 counts the option bytes still to read, so the walk never reads payload. Every byte is extracted (into `hdr.tcp_opt`), so the deparser re-emits the options unchanged and this walk needs no second pass. EOL is a one-byte option and the walk continues: FoxIO (through tshark) lists every padding byte after EOL as another 0.

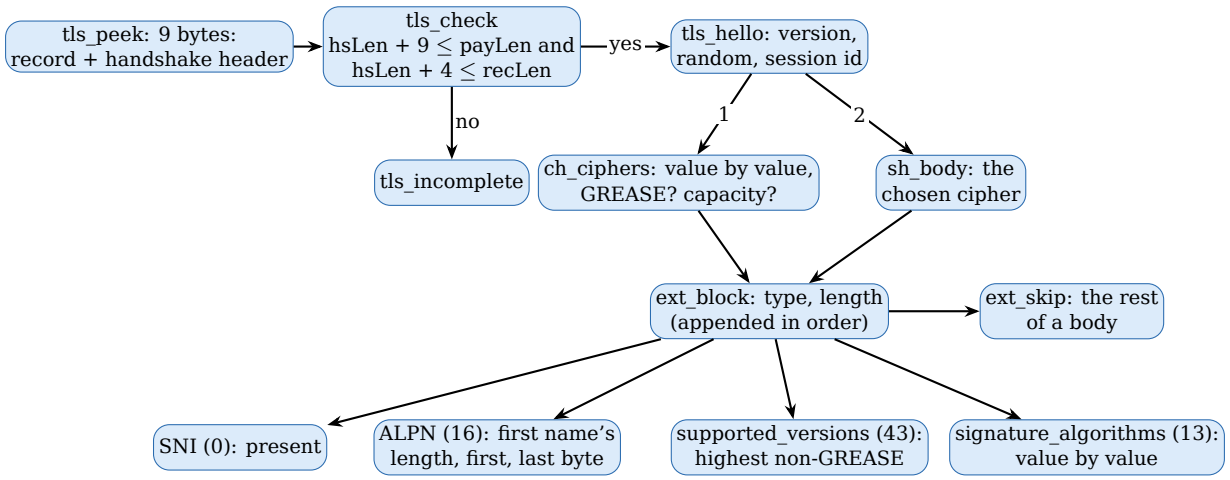


Figure 8: TLS hello walk (`parser_tls.p4`). Each list value is read with `lookahead` and skipped with `advance`; a GREASE value (`0x?a?a`, equal bytes) goes back to the loop without being kept; a value beyond the capacity sets an overflow bit. Every extension type is appended in order (client: without GREASE).

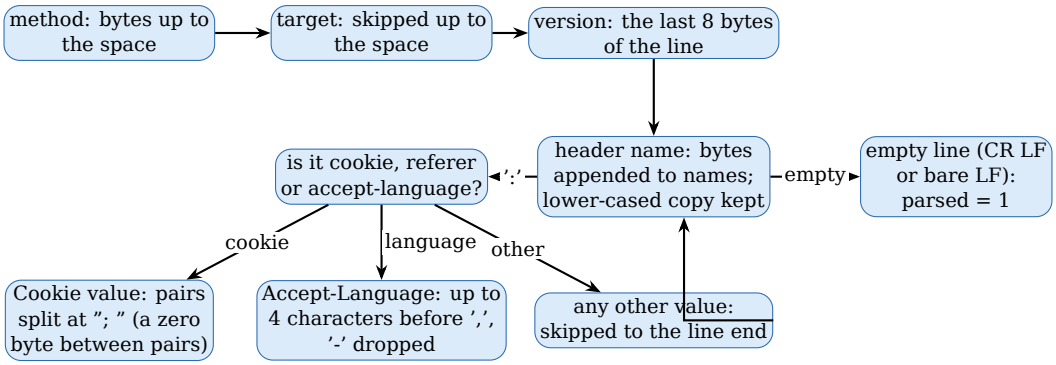


Figure 9: HTTP request walk (`parser_http.p4`), one byte per state. A name is appended to the names list as it is read; when it turns out to be Cookie or Referer (excluded from JA4H's name list), the last 6 or 7 bytes are removed with a constant-width slice. If the payload ends before the empty line, `incomplete` is set instead.

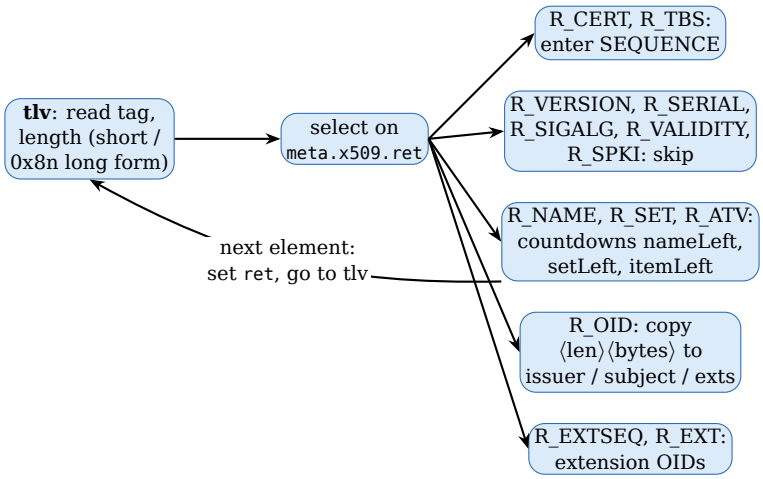


Figure 10: DER walk (`parser_x509.p4`). A P4 parser has no calls, so one shared `tlv` reader is entered with a "return code" and a `select` on it continues in the right state. Nesting ends are known from byte countdowns per level, so no stack is needed.

2.3 Ingress: the order of the controls

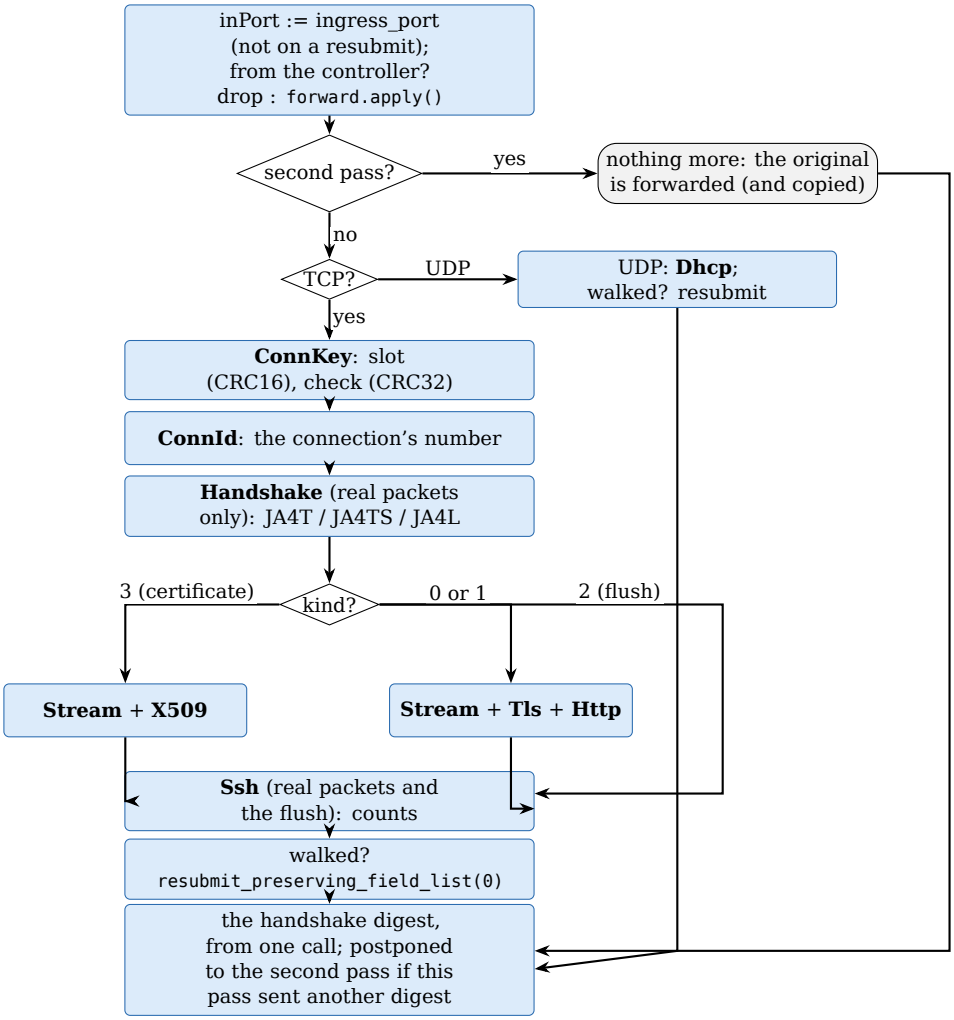


Figure 11: `MyIngress` in `main.p4`. "kind" is the packet-out kind of a packet from the controller (0 for a real packet): 1 a joined message, 2 an SSH flush (Ssh only), 3 a certificate. Ssh runs after Stream, which sets the packet's direction (client or server) that Ssh needs on ports other than 22.

Why: Every control is applied exactly once and in this order because (a) BMv2 allows a table / control instance to be applied in one place only, (b) Stream must see the parse result (incomplete, parsed) before Tls/Http decide to report, and (c) BMv2 keeps one digest per packet: the handshake digest is therefore taken out of Handshake and sent at the end, from one call, and postponed to the second pass when this pass sends a message digest (the handshake's ACK can carry a whole ClientHello) or Ssh's digest (on the SYN of a reused port, the previous connection's last window; on an SSH banner piggybacked on the handshake's ACK).

2.4 Per-connection state

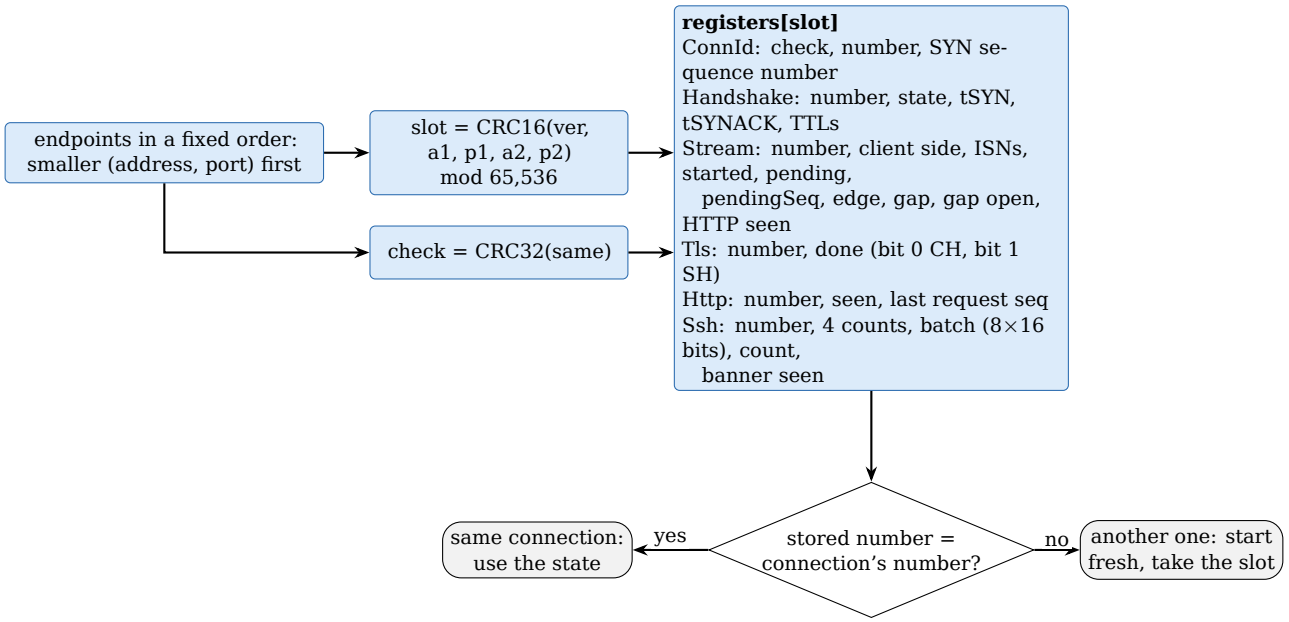


Figure 12: ConnKey and ConnId (`main.p4`) and the registers. Both directions of a connection find the same slot. ConnId gives the connection a number: a new one when another connection (a different check) comes to the slot, and when a SYN with a new initial sequence number arrives on the same endpoints (port reuse); a repeated SYN keeps it. Every control keeps its state with this number, and every digest and copy carries it.

Why: A P4 switch cannot insert table entries itself, so per-connection state lives in register arrays indexed by a hash. CRC16 gives the index; the independent CRC32 tells two connections in one slot apart with a 1 in 2^{32} chance of a false match.

Why: The connection number: the endpoints alone do not identify a connection. A client, and much faster a NAT, reuses addresses and ports for a new connection after the old one closed. Before the number, the second connection's fingerprints were dropped by the controller (it kept the first value per endpoints) and SSH counts of the two connections were added together into a wrong JA4SSH (found by `test_port_reuse`).

Why not something else: The controller installs one table entry per connection: each new connection would wait for a round trip to the controller before its SYN state exists, and the SYN's timestamp (JA4L) would be lost. A larger hash: 65,536 slots is plenty for one link of a test network; the size is one constant.

3 Switch and controller: every channel between them

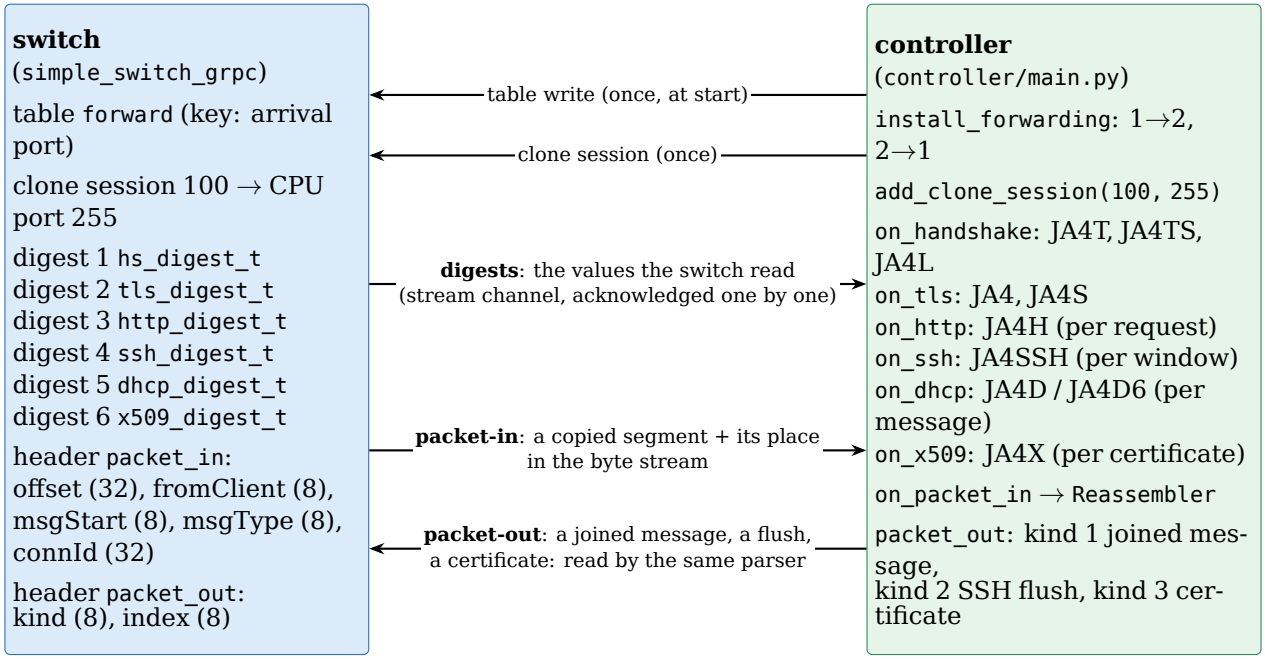


Figure 13: The interface. Everything goes over one P4Runtime gRPC connection (`controller/p4runtime_client.py`): the controller becomes primary (arbitration), loads `build/main.json` with its P4Info, writes the table entries and the clone session, enables the six digests, and then only listens and answers.

3.1 The six digests

id	struct	sent when	main fields	controller writes
1	`hs_digest_t`	first SYN (JA4T), first SYN-ACK (JA4TS), the ACK after it (JA4L)	window, option kinds (up to 40), MSS, window scale, two latencies, two TTLs, connId	`handshake.py`: `64240_2-4-8-1-3_1460_7, 1029_64`
2	`tls_digest_t`	a complete hello, once per connection and direction	versions, SNI flag, ciphers (≤128), extensions (≤64), signature algorithms (≤64), ALPN first/last, overflow, connId	`tls.py`: sort, SHA-256 → `t13d1516h2_…`
3	`http_digest_t`	a complete request header block (not twice for one sequence number)	method, version, cookie / referer flags, header count, language, names (≤512 B), cookie pairs (≤1,024 B), serial, connId, the request's sequence number (the controller lists a connection's requests in stream order)	`http.py` → `ge11cr04engb_…`
4	`ssh_digest_t`	8 lengths collected; window of 200 closed; flush; an earlier connection's last window on a reused port's SYN; an SSH banner on a port other than 22	batch of 8 (direction + length), 4 counts, serial, connId, direction	`ssh.py`: modes → `c36s52_c100s100_c0s0`
5	`dhcp_digest_t`	every DHCP / DHCPv6 message	type, size, i/d flags, options (≤64), requested codes (≤128), serial	`dhcp.py` → `disco0576nn_…`
6	`x509_digest_t`	every certificate sent back by the controller	chain index, issuer / subject / extension OIDs (≤256 B each), connId	`x509.py`: hex, SHA-256 → `a373a9f83c6b_…`

Digest settings (`enable_digest`): `max_list_size = 1` and `max_timeout_ns = 0` (send each digest

at once, alone), `ack_timeout_ns` = 1 s. The controller acknowledges every list as it handles it. BMv2 holds an unacknowledged digest in a filter and drops an identical one: the DHCP, HTTP and SSH digests carry a serial number for that reason (Section 7.10). Every digest of a TCP connection carries the connection's number (`connId`, ConnId in `main.p4`): the controller keeps one record per (endpoints, number), so a new connection on reused ports gets its own record.

3.2 A message split over segments, arriving out of order

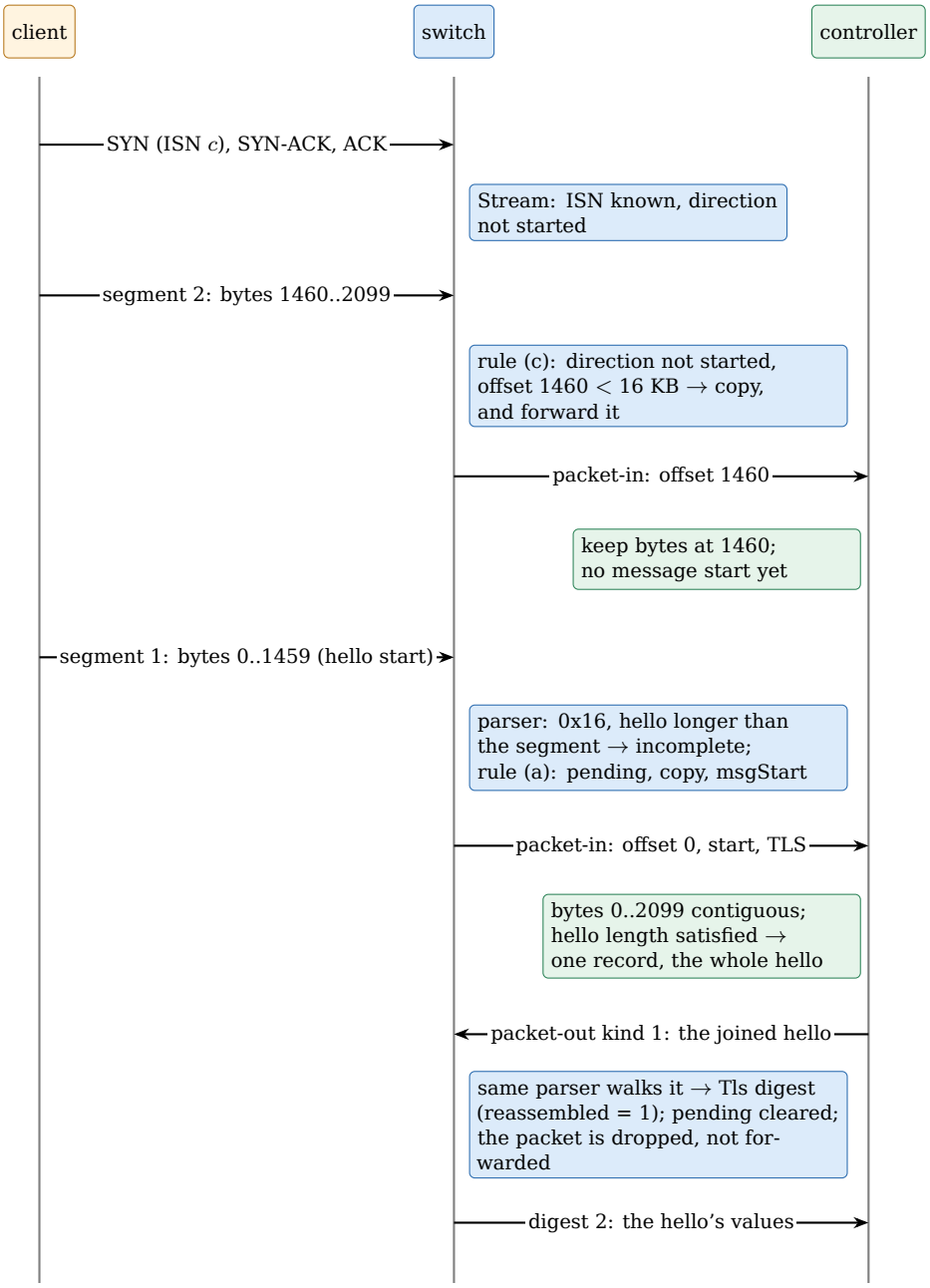


Figure 14: The out-of-order case end to end. The original segments are forwarded unchanged as they pass; only copies go to the controller. The controller joins bytes by TLS framing only and never reads a field: the fingerprint is still read by the switch's parser.

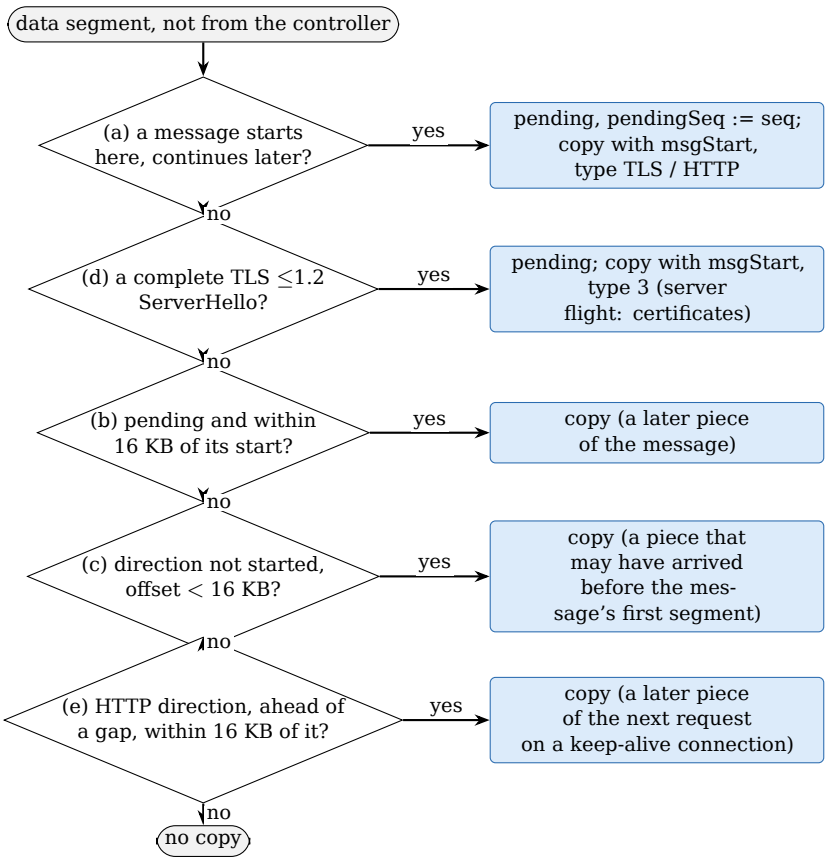


Figure 15: `p4/stream.p4`: which segments the controller must see. Offsets are `seq` − ISN − 1 per direction (ISN from the SYN / SYN-ACK; without them, learned from the first message start). A joined message (or the certificates) coming back clears `pending`. For (e) the switch keeps, per direction, the edge (the sequence number after the furthest byte seen): a segment beyond it opens a gap at the edge; the gap closes when a segment covering its first byte arrives, and is given up 16 KB on. Everything else is forwarded without a copy: the controller sees only what a split message may need.

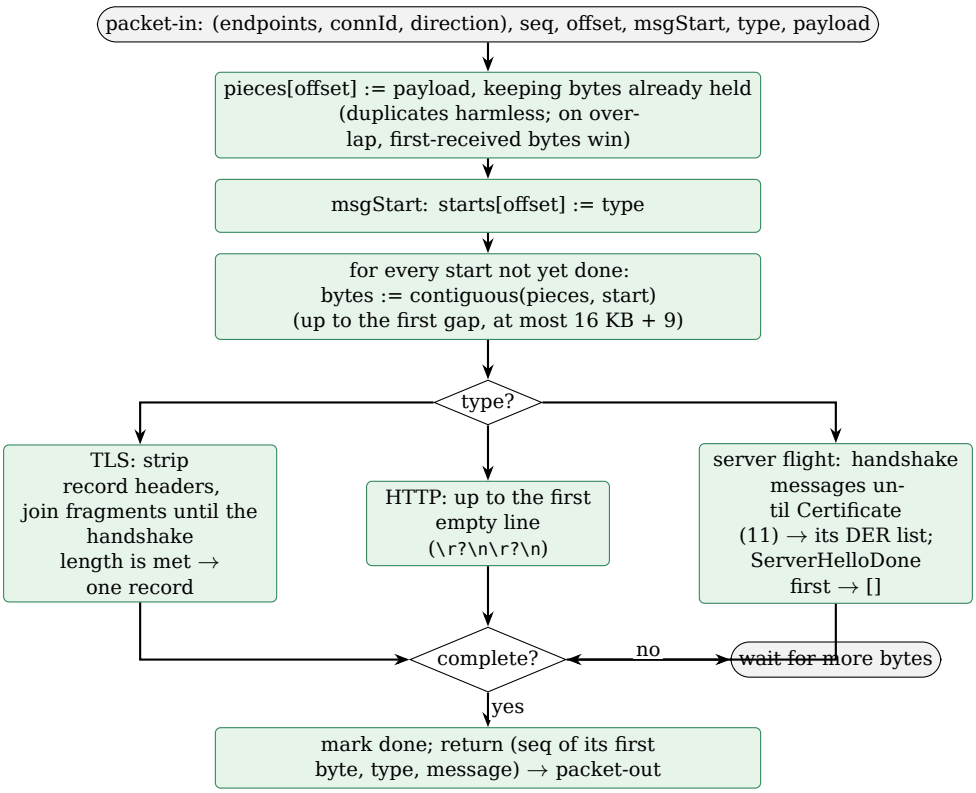


Figure 16: `controller/reassembly.py`. It only knows framing (record headers, handshake lengths, the empty line, DER lengths of the certificate list) and never interprets a fingerprint field. A "start" flagged on a short segment that only looked like a message never completes and is ignored. Bounded: at most 32 KB per direction (lowest offsets dropped first), a direction idle for 60 s is dropped, at most 100,000 directions. The bytes of a joined message are not dropped at once: a false start in the middle of a request can be "joined" before the real one, whose first segment arrives last, and that one still needs them.

3.3 TLS 1.2 certificates (JA4X) and the SSH flush

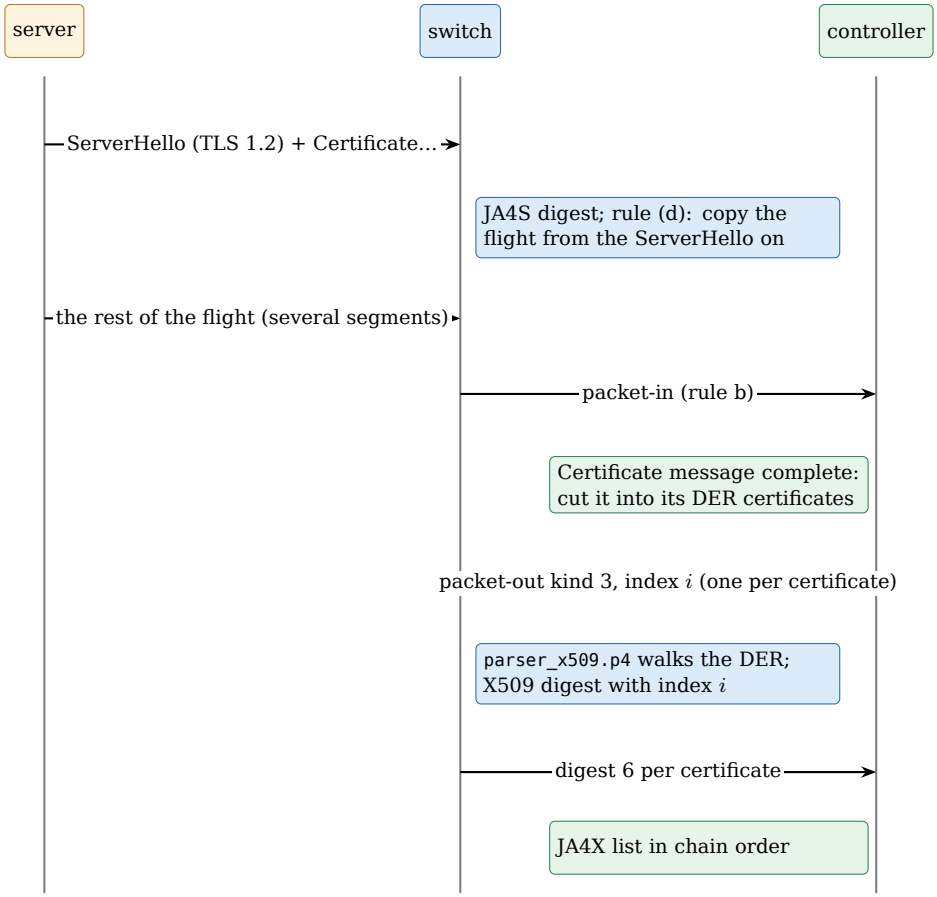


Figure 17: JA4X. Certificates are long (1–2 KB each) and always span segments, so they always take the join path.

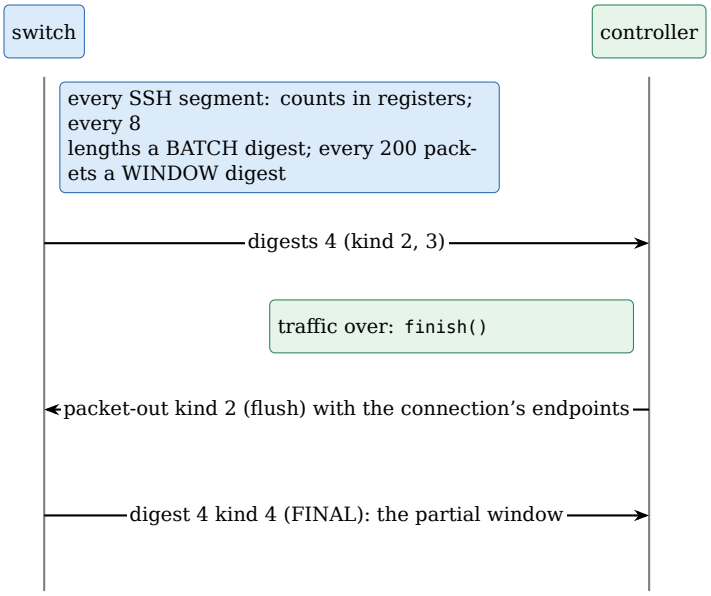


Figure 18: The last JA4SSH window. FoxIO reports the partial last window at the end of its capture; P4Runtime on BMv2 cannot read registers (UNIMPLEMENTED), so the switch is asked to report it itself.

4 Each fingerprint, end to end

Every fingerprint follows the same three steps: **the switch reads** (parser + control, state in registers), **sends a digest**, **the controller writes the text**. The figures show each step and the anatomy of the result.

4.1 JA4T, JA4TS, JA4L: the TCP handshake

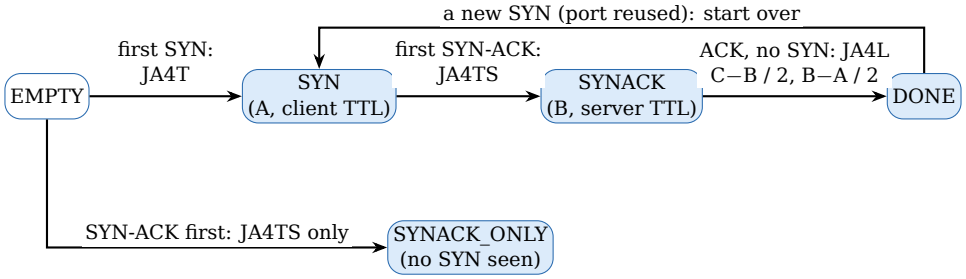
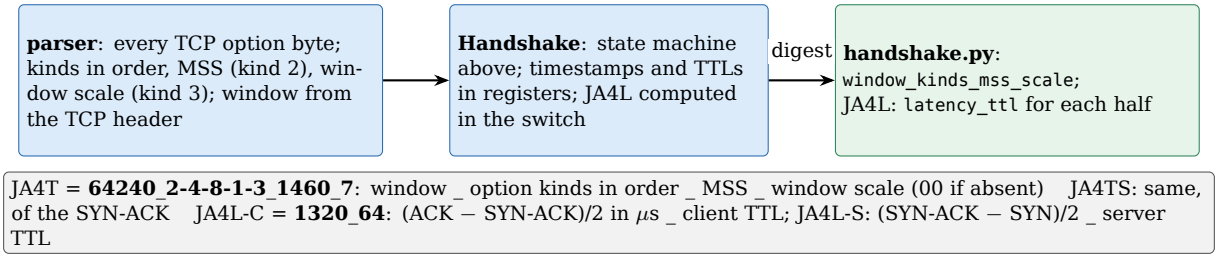
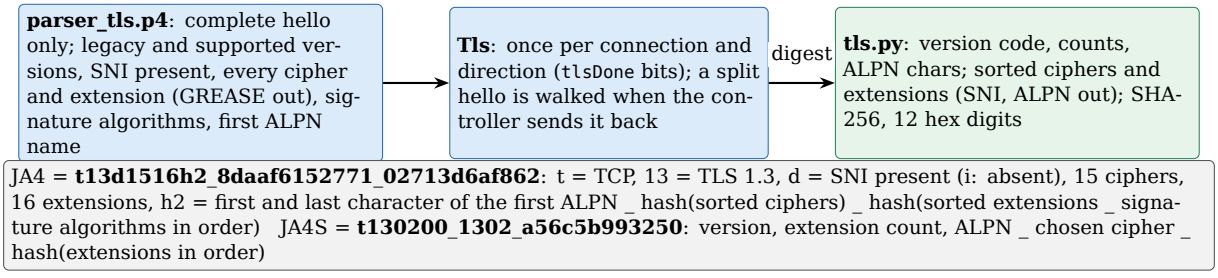


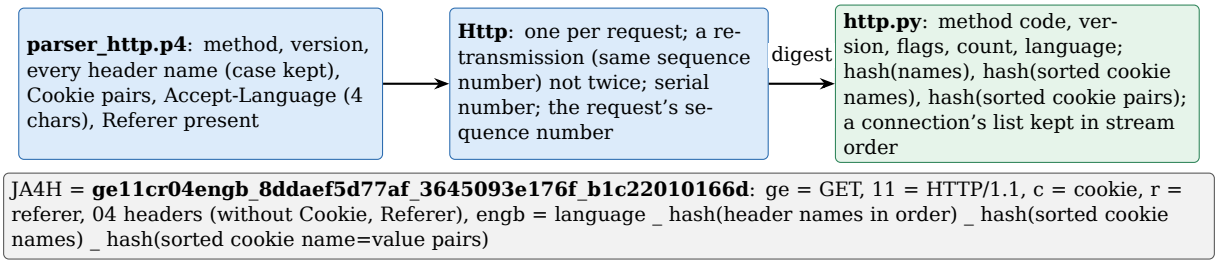
Figure 19: `p4/handshake.p4`: one state per connection slot. A repeated SYN or SYN-ACK changes nothing (the reference also takes the first). Time = the switch's ingress timestamp in microseconds.



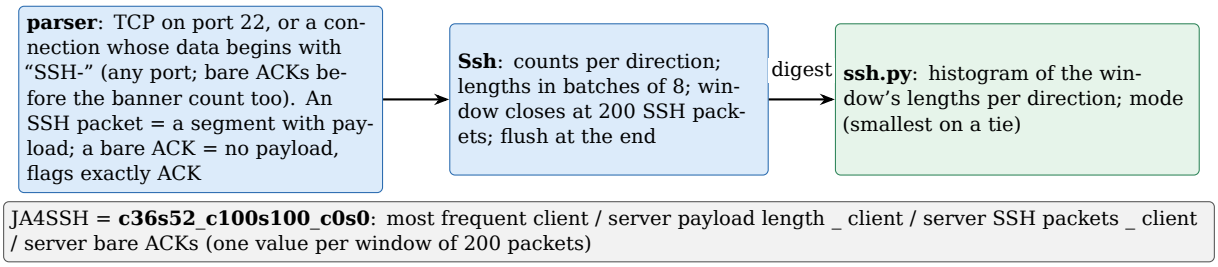
4.2 JA4 and JA4S: TLS hellos



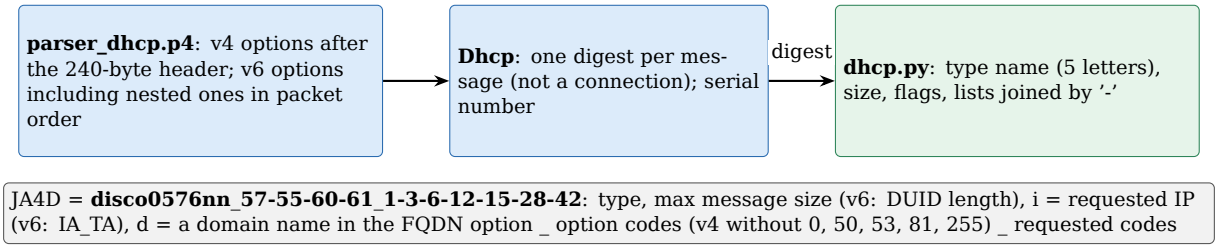
4.3 JA4H: HTTP requests



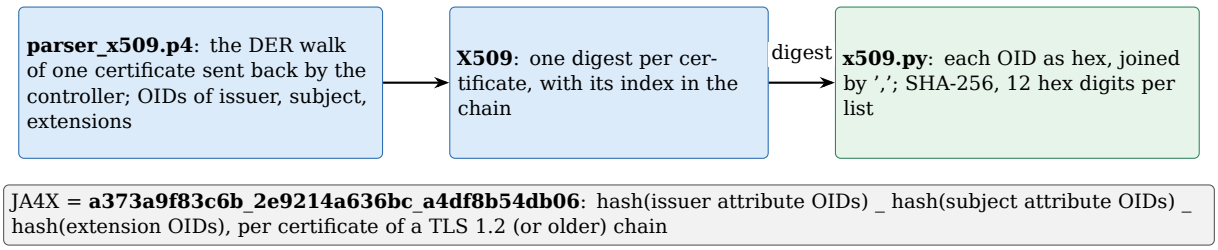
4.4 JA4SSH: SSH packet sizes



4.5 JA4D and JA4D6: DHCP



4.6 JA4X: certificates



5 How correctness is checked

5.1 The replay bench (no root)

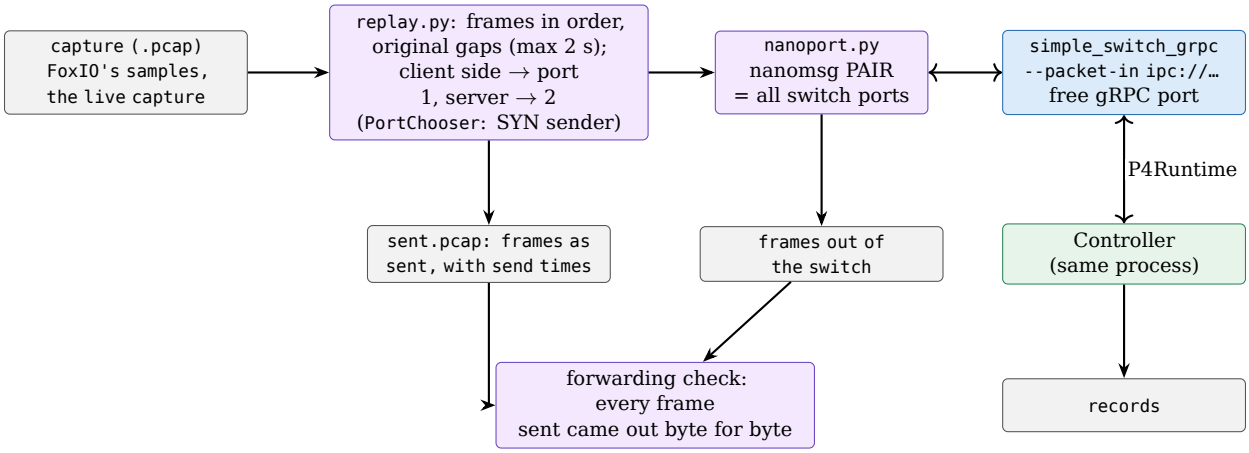


Figure 20: `tools/replay.py`. The real switch binary and the real controller, without network namespaces: BMv2's `--packet-in` option makes one nanomsg socket stand for all its ports. A warm-up frame goes first (the first frame after loading the program reaches the pipeline late). The switch dies with its parent (`PR_SET_PDEATHSIG`), so no run can leave one behind.

5.2 Comparison against independent references

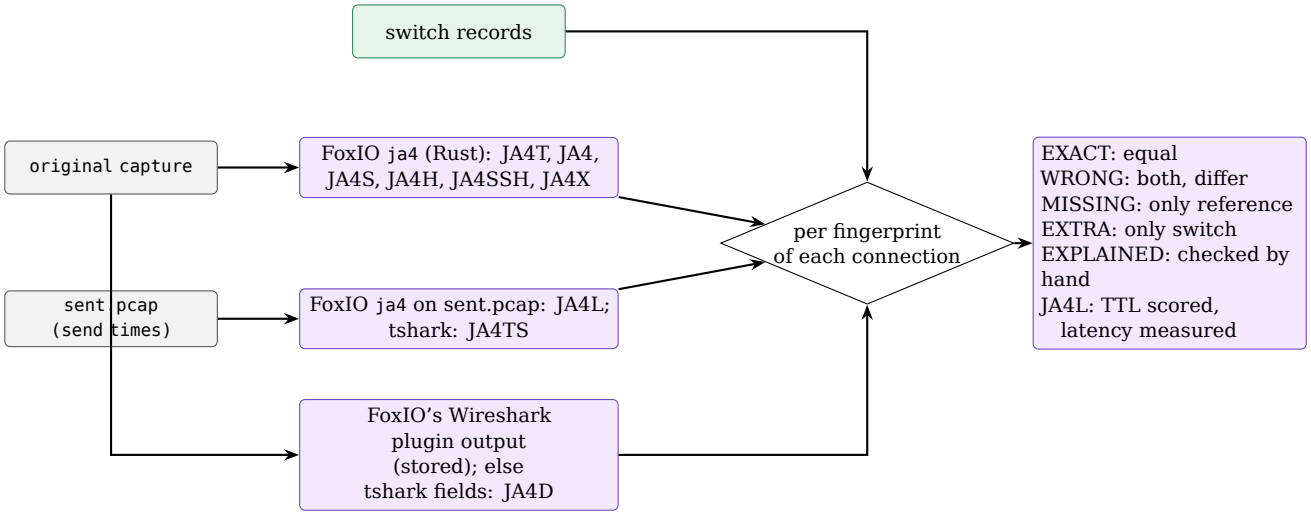


Figure 21: `tools/compare.py` with `tools/reference.py`. JA4 values come from the original capture (timing-free); JA4L and JA4TS from the frames as the switch received them, so both sides time the same packets.

5.3 The live run (root)

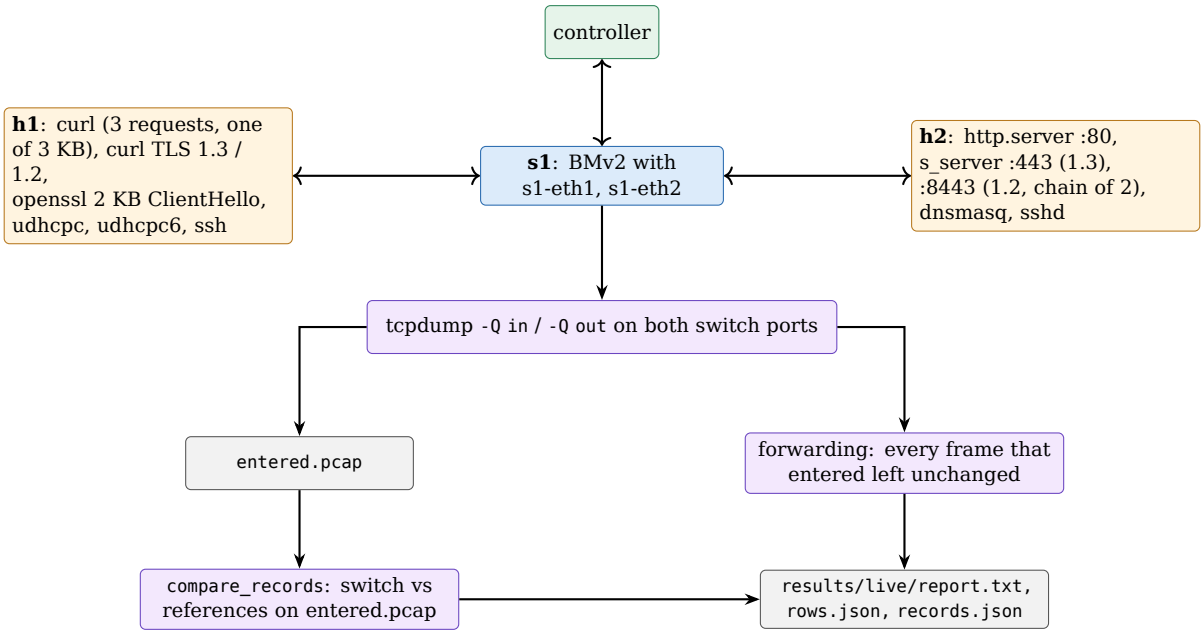


Figure 22: `network/run_live.py`: real programs, real TCP stacks (offloads off, so TCP itself splits the large messages), the same switch and controller. The clients start only when h2 listens on every server port; every client runs under `timeout` and its exit status is checked (a failed step is reported); every server's process id is kept and only those processes are stopped. `tools/live_tex.py` then writes the results table from `rows.json`.

5.4 The tests: what each one protects

test	what it sends and expects	protects
`test_split_reorder`	real hellos and requests cut into 2–5 pieces: in order, reversed, every permutation of 3, duplicated, overlapping, after 20 other bytes, without the SYN; expected = FoxIO on the message sent whole. 202 cases exact; 8 known limit cases must never be wrong	Stream, Reassembler, the parsers on joined messages
`test_server_flight_reorder`	h2's TLS 1.2 flight from the live run (ServerHello, chain of 2, 2,241 bytes) cut as TCP cut it, in every order of 3 pieces, in 100-byte pieces reversed, duplicated, overlapping; expected = FoxIO on the flight sent whole. 19 cases exact; a split ServerHello: JA4S exact, JA4X never wrong	rules (c), (d), (b) for the server; the flight's join
`test_keepalive_reorder`	on one connection: a whole request, a second request cut into 2 or 3 pieces in 18 orders, a third whole; expected = FoxIO's three JA4H on the requests sent whole, in order. 18 cases exact	rule (e); JA4H in stream order
`test_port_reuse`	two TLS connections and two SSH sessions, each pair on one four-tuple (FIN, then a new SYN; one SYN repeated); compared with FoxIO by stream	ConnId; records per connection; SSH window on reuse
`test_vlan_and_ssh_port`	TLS in one 802.1Q tag, HTTP in 802.1ad + 802.1Q, the live SSH session on port 2222; compared with FoxIO	VLAN parsing; SSH by banner
`test_controller_state`	reassembler alone, with a clock passed in: 300 split requests on one connection, an idle direction, 1,000 directions, 100 KB without a start, a false start inside a request	D24 bounds; no early pruning
`test_ja4l_gaps`	a handshake with 1 s and 0.5 s gaps; JA4L within 2%; JA4T/JA4TS exact	Handshake arithmetic (halving, order)
`test_one_digest_per_packet`	a whole ClientHello / HTTP request on the handshake's ACK: JA4L and the message's fingerprint both reported	the deferred handshake digest
`test_identical_digests`	3 identical DHCP offers, 3 identical HTTP requests, 4 identical SSH windows (acknowledgements held back)	the serial numbers
`test_dhcp6_fqdn`	option 39 with an empty and a real name: `n`, `d`	the JA4D6 domain flag
`test_capacity_limits`	lists at and one past each capacity: exact, then "incomplete"	every overflow path

5.5 The measurement study

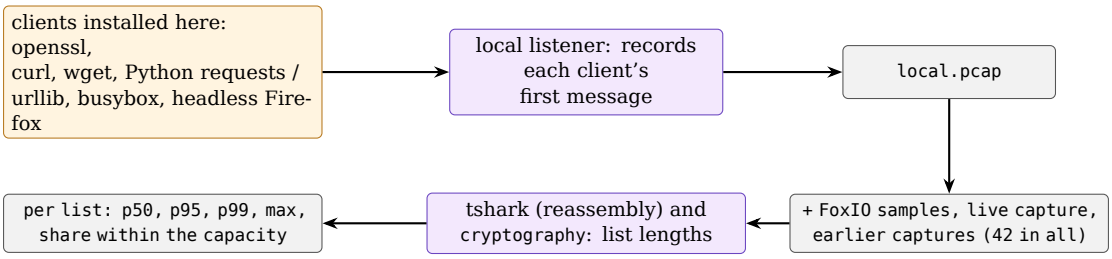


Figure 23: `tools/measure_field_counts.py`: are the switch's capacities large enough? (Results: Section 12.)

Part II

Explanation

6 The problem and the scope

Goal (as set by the faculty). A linear network with one switch and two hosts (h1 – s1 – h2). Traffic flows between the hosts; the switch computes every JA4+ fingerprint it can, with P4, correctly, including when a message is split over several packets or its packets arrive out of order ("the server's 2nd packet may come first"). A controller is part of the design, so that higher-level programs can drive the switch. Nothing else: no attack mitigation, no flood protection, no hardware port. The contribution: JA4+ fingerprinting done in the P4 data plane.

What is fingerprinted. Before a protocol's session is encrypted (or for protocols that are never encrypted), the two sides exchange clear-text messages whose structure depends on the software that sent them:

Fingerprint	clear-text exchange read	per	reference
JA4T / JA4TS	client SYN / server SYN-ACK: window, TCP options, MSS, window scale	connection	FoxIO `ja4`; tshark
JA4L-C / -S	timing of SYN, SYN-ACK, ACK and the TTLs	connection	FoxIO `ja4`
JA4 / JA4S	TLS ClientHello / ServerHello (all versions; the hellos are always clear)	connection	FoxIO `ja4`
JA4H	HTTP/1.x request header block	request	FoxIO `ja4`
JA4SSH	sizes and directions of SSH packets (SSH payload is encrypted after key exchange)	200 packets	FoxIO `ja4`
JA4D / JA4D6	DHCP / DHCPv6 messages (never encrypted)	message	FoxIO's Wireshark plugin
JA4X	certificates in a TLS 1.2 (or older) server flight	certificate	FoxIO `ja4`

Out of reach, by nature: HTTP/2 and HTTP/3 requests and TLS 1.3 certificates are inside encryption (FoxIO's tool computes them only for captures that store the session keys). **Not implemented, by scope:** JA4 over QUIC (its ClientHello is in Initial packets whose keys are derived from public values: readable passively, but v1model has no AES or HKDF, so it would need the controller to decrypt), JA4 over DTLS, tunnels (GRE, VXLAN, Geneve: the inner headers would need a second parser path), IPv6 extension headers. Section 13 lists what is done and what is not, with the reason and what doing it would take.

Target. BMv2 (`simple_switch_grpc`), the P4-16 v1model architecture, P4Runtime over gRPC. BMv2 is the reference software switch of P4; a hardware target (Tofino) has different limits and is not part of this project.

What "correct" means here. For every fingerprint, the switch's value equals the value FoxIO's own tools compute on the same frames, character for character; and every frame leaves the switch unchanged. JA4L is a time measurement, so its TTL part must be equal and its latency is reported as a difference distribution (BMv2 timestamps a packet when its pipeline takes it up, which a software switch delays by microseconds to milliseconds).

7 Design decisions: what, why, and why not the alternatives

Each decision is stated, then the reason, then the alternatives that were considered and why they were not taken.

7.1 D1. BMv2, v1model and a P4Runtime controller

Decision. The switch is BMv2's `simple_switch_grpc` running a P4-16 v1model program; a Python controller talks to it over P4Runtime.

Why. BMv2 is the reference P4 target, runs anywhere, needs no hardware, and supports everything the design uses: parser loops with `lookahead`/`advance`, registers, hashes, digests, resubmit, clone, packet-in/out. P4Runtime is the standard control interface, so "a high-level program drives the switch" means exactly: another P4Runtime client, or this controller's Python API.

Why not something else: Tofino: hardware limits (parser depth, stages, no 2,048-bit fields) would dominate the work; the faculty asked for fingerprinting, not a hardware port. eBPF/XDP: not P4. Other P4 architectures: v1model is the one `simple_switch_grpc` implements completely (resubmit, clone with preserved metadata, digests); PSA support in BMv2 is a separate, less complete switch, and TNA is Tofino's.

7.2 D2. The parser reads every fingerprint value

Decision. All variable-length reading (TCP options, TLS lists, HTTP headers, DHCP options, DER) is done by P4 parser states. The controller never parses a fingerprint field.

Why. This is the contribution: fingerprinting in the data plane. The parser is the only P4 block that can loop over a variable number of items (a state may transition to itself).

Why not something else: Send every packet to the controller and parse there: a software fingerprinter with a switch in front. Parse with match-action tables: tables match fixed offsets; variable-length lists need the parser's loops.

7.3 D3. Results leave the switch as digests

Decision. Each control sends its values in one digest struct (six kinds), as soon as they are complete.

Why. A digest carries only the extracted values (a few hundred bytes), asynchronously, without the packet; the packet is forwarded at once. It is P4Runtime's standard channel for data-plane-to-controller events.

Why not something else: Packet-in of every interesting packet: sends the whole packet and makes the controller parse. Controller reads registers: BMv2's P4Runtime server answers UNIMPLEMENTED for register reads (found, see the problems table), and polling would race with new connections. Write results into a header and mirror: needs a custom receiver and still a clone per message.

7.4 D4. Sorting, SHA-256 and text in the controller

Decision. The switch sends lists in packet order; the controller sorts the cipher and extension lists, hashes with SHA-256 and writes FoxIO's text.

Why. v1model has no SHA-256 extern, no strings, and no loops in the pipeline (only in the parser). The fingerprint is fully determined by the values the switch extracted; sorting and hashing them anywhere gives the same result.

Why not something else: A sorting network in P4 (bitonic, as in the earlier ja4p4 plan): needed on a Tofino to report sorted lists from the data plane; on BMv2 it adds hundreds of compare-and-swap actions and proves nothing about fingerprinting. A CRC instead of SHA-256: would not be JA4 and could not be compared with FoxIO.

7.5 D5. Walked packets are resubmitted and forwarded on a second pass

Decision. A parser walk skips bytes with `advance`; the packet is then resubmitted, and on the second pass only Ethernet is parsed, so the original bytes are forwarded.

Why. `advance` removes the skipped bytes from what the deparser emits. Resubmit runs the original packet through the pipeline again, with chosen metadata preserved (field list 0: arrival port, the pending handshake record).

Why not something else: Extract every byte into headers: thousands of header-stack entries; the deparser would re-emit them (the TCP option walk does extract, since options are at most 40 bytes). Recirculate: passes through egress and the deparser, which would emit the shortened packet.

7.6 D6. Split and out-of-order messages are joined by the controller and read by the switch

Decision. The switch decides which segments may hold pieces of a message (Fig. 15) and copies them with their stream offset; the controller orders the bytes by framing only and sends the complete message back; the same parser reads it.

Why. A P4 switch processes one packet at a time and cannot hold a packet back or keep kilobytes of payload per connection. Joining needs a buffer; reading does not. So the buffer is in the controller and the reading stays in P4. The controller's join is framing only (record headers, the empty line, DER lengths), never a fingerprint field.

Why not something else: Buffer segments in registers: a ClientHello reaches 2 KB, certificates many KB; registers sized for that per connection are impossible on hardware and costly on BMv2, and register access is per packet, fixed-width. Parse each piece separately and merge partial states: a list value can be cut in the middle and the order of pieces is unknown; the state machine would need to be resumable at every byte in both orders. Ignore split messages: wrong by omission for exactly the cases the faculty named.

7.7 D7. Copies are made at egress (E2E clone)

Decision. `clone_preserving_field_list(CloneType.E2E, 100, 1)` in egress; the clone gets the `packet_in` header.

Why. An ingress-to-egress clone is parsed again from the start, and its walk consumes the bytes again: the copy arrived without payload (found by the tests). At egress the packet has all its bytes.

7.8 D8. Per-connection state in hashed register slots

Decision. ConnKey orders the endpoints, hashes them (CRC16 slot, CRC32 check); every control keeps its state in register arrays of 65,536 entries indexed by the slot, together with the connection's number (D21).

Why. A P4 program cannot add table entries; registers are the data plane's writable memory. Ordering the endpoints makes both directions find the same slot. The 32-bit check tells two connections in one slot apart.

Why not something else: One table entry per connection from the controller: a round trip before the first packet's state exists, and the SYN's timestamp would be lost. Bigger slots / buckets: unneeded for one link; one constant.

7.9 D9. One `digest()` call per control; the handshake digest sent last

Decision. Every control has exactly one digest call; the handshake record is stored in metadata and sent from a single call at the end of ingress, on the second pass if this pass sent a message digest.

Why. p4c makes one learn list per call site (a second call site of the same struct is a different list the controller did not enable), and BMv2 keeps one digest per packet: the handshake's ACK can carry a whole ClientHello, and its JA4L would be lost (found with SSH, then covered by `test_one_digest_per_packet`).

7.10 D10. A serial number in DHCP, HTTP and SSH digests

Decision. A 32-bit counter register per digest kind; each digest carries its value.

Why. BMv2 keeps every digest until it is acknowledged and drops a new one identical to it. Identical messages close together (DHCP retransmissions and their answers, the same HTTP request repeated, identical SSH windows of a bulk transfer) were lost. 32 bits: the serial need only differ among digests unacknowledged at the same time (rate $\times$ acknowledgement time, below 2^{20} even at 10^6/s for 1 s); wrapping at 2^{32} is harmless and needs no flag, because the controller never reads the value.

Why not something else: Acknowledge faster: the filter exists until the acknowledgement arrives, however fast. Use the timestamp: two packets in one microsecond collide. Per-connection counters: DHCP has no connection.

7.11 D11. GREASE is recognised in the parser's `select`

Decision. `IS_GREASE(v)`: low nibbles of both bytes are 0xA and the two bytes are equal; the value is skipped before it is stored.

Why. GREASE is a bit pattern, not a lookup; a `select` on it costs nothing and keeps the lists clean, so counts are right without a second pass.

Why not something else: A match-action table marking slots invalid: needs a list of slots to filter afterwards and then compaction; the parser simply never stores them.

7.12 D12. Capacities: 128 / 64 / 64 values, 512 / 1,024 / 256 bytes, then "incomplete"

Decision. Each list is kept in fixed-width fields (at most 2,048 bits each, chained); past its capacity the count stays right, an overflow bit is set, and the fingerprint is reported as incomplete.

Why. A wrong fingerprint is worse than none. The capacities were checked by a measurement study (every one of 1,238 measured lists fits; closest: cookies 863 of 1,024 bytes, ciphers 86 of 128) and the overflow path by `test_capacity_limits`.

Why not something else: Larger capacities: cost digest bits and parser iterations for no measured case. Truncate silently: would give wrong hashes.

7.13 D13. JA4L timed in the switch

Decision. The switch stores the SYN and SYN-ACK times and TTLs and computes both halves when the ACK passes.

Why. The point of JA4L is the network position of the measuring device; the switch is that device. The controller would see digests later and with its own jitter.

Why not something else: Controller time: measures the control channel, not the network. Limit kept honest: BMv2 stamps when its pipeline takes the packet up, so the latency is reported as a measurement and the arithmetic is tested with known gaps.

7.14 D14. SSH: counts in the switch, the mode in the controller

Decision. The switch counts SSH packets and bare ACKs per direction and window exactly, and sends the payload lengths in batches of 8; the controller computes the most frequent length.

Why. A mode needs a histogram over arbitrary lengths; a data plane has no dynamic map. Everything else of JA4SSH is counting, which registers do exactly.

Why not something else: A fixed histogram in registers: lengths range up to 64 KB; a per-connection histogram is impossible, and approximate buckets would give a different fingerprint.

7.15 D15. The last SSH window through a flush packet

Decision. At the end, the controller sends a packet-out of kind 2 with the connection's endpoints; the switch reports the partial window itself.

Why. FoxIO's tool reports the partial last window; BMv2's P4Runtime cannot read registers.

7.16 D16. JA4X: the controller cuts certificates out, the switch walks DER

Decision. The controller follows TLS framing to the Certificate message and sends each DER certificate back (kind 3, with its index); `parser_x509.p4` reads the OIDs.

Why. Certificates always span segments, so they need the join path anyway; the DER walk (the fingerprint's reading) is in P4, with one shared TLV reader and return codes because a parser has no calls.

7.17 D17. Byte-per-state HTTP walk with chained 2,048-bit fields

Decision. The request is read one byte per parser state; lists grow through chains of 2,048-bit fields by constant-width slices.

Why. HTTP has no length fields: the end of each name, value and line is found byte by byte. BMv2 allows at most 2,048 bits per field and only short variable shifts, so a list longer than 256 bytes must be a chain grown by constant slices.

7.18 D18. A rootless replay bench next to the live network

Decision. `tools/replay.py` runs the real switch and controller with BMv2's nanomsg packet socket as its ports.

Why. Hundreds of runs (every capture, every split case) without root or Mininet; the same binary and controller as in the live run. The live Mininet run then confirms the result on real TCP stacks.

7.19 D19. FoxIO's own tools as the reference

Decision. The Rust `ja4` tool for most fingerprints, tshark fields for JA4TS (not in the open tool), FoxIO's Wireshark plugin output (or the same rule on tshark fields) for JA4D.

Why. Independent of this project: a match with the authors' implementation is the strongest evidence of correctness.

Why not something else: Compare with my own Python implementation: two implementations of my own reading could share the same misunderstanding.

7.20 D20. One design only

Decision. The two approaches discussed at the start (pure P4 without external programs; P4 invoking high-level code) were set aside by the faculty; the design is one switch program plus one controller.

Why. Focus on correct fingerprints first.

7.21 D21. A number for every connection (port reuse)

Decision. Control `ConnId` (`main.p4`) gives each connection a number from a counter: when another connection comes to the slot (a different check), and when a SYN with a new initial sequence number arrives on the same endpoints. Every control keeps its state with this number instead of the check; every TCP digest and every copy carries it; the controller keys records, the reassembler and SSH windows by (endpoints, number); the comparison keys FoxIO's records by (endpoints, place among the TCP streams on those endpoints).

Why. Endpoints do not identify a connection: clients and NATs reuse ports. Before, the controller kept the first value per endpoints (the second connection's JA4 was lost) and the SSH counts of two connections were added together into a wrong value. A repeated SYN (same sequence number) must not start a new connection.

Why not something else: Key by the client's initial sequence number in the controller: only the SYN carries it; every digest would need it, and connections without a SYN seen have none. Detect a new connection in the controller from a new JA4T: digests of different kinds arrive in no fixed order. Key on FIN / RST: they can be lost or missing from a capture; a SYN with a new sequence number is what tshark and FoxIO use.

7.22 D22. Rule (e): pieces ahead of a gap in an HTTP direction

Decision. Per direction the switch keeps the edge (after the furthest byte seen). In a direction that has carried an HTTP request, a segment beyond the edge opens a gap, and segments within 16 KB of the gap's start are copied until a segment covering the gap's first byte arrives. The controller lists a connection's JA4H by each request's sequence number.

Why. Rule (c) covers only a direction's first message. On a keep-alive connection the second request, cut in two and second piece first, was never reported. Reporting in stream order is FoxIO's order; a joined request otherwise arrives after the whole requests that followed it.

Why not something else: Copy every segment after a gap in every connection: TLS and SSH connections carry nothing more to read after their hellos, so copies there are only load. Track every hole: one gap per direction covers the case at the cost of four registers; further holes inside a pending message are covered by rule (b).

7.23 D23. VLAN tags and SSH on any port

Decision. The parser reads up to two VLAN tags (802.1Q, 802.1ad outer). A connection is SSH when it is on port 22 or a side's data begins with "SSH-"; bare ACKs are counted from the start of every connection, so those before the banner count.

Why. FoxIO's tool reads through VLAN tags and recognises SSH by its banner on any port (tshark's heuristic): the switch reported nothing on tagged frames and on SSH on port 2222. FoxIO counts the handshake's ACK of such a connection as a client bare ACK (the first version gave `c6s6` for FoxIO's `c7s6`).

Why not something else: A list of SSH ports from the controller: misses the unknown ones, and FoxIO needs no list. More than two tags: not used on access links; such frames are forwarded, not fingerprinted.

7.24 D24. Bounded controller buffers

Decision. The reassembler keeps at most 32 KB per direction (lowest offsets dropped first), drops a direction idle for 60 s, and keeps at most 100,000 directions.

Why. Without bounds a long run kept every copied piece forever, and half-sent messages could fill the controller.

Why not something else: Drop a message's bytes as soon as it is joined: tried, and it broke 3 of the 202 split cases. A one-byte segment in the middle of a request that begins like a request is "joined" first; the real request, whose first segment comes last, then lacked those bytes.

8 Every code artifact

One card per file: its role, where it sits (Fig. 3), what it takes and gives, the logic it follows, why it is needed, the alternatives, and what checks it. Line counts are those of the current version.

8.1 Data plane (P4)

`p4/headers.p4` (323 lines)

Role	every header and metadata type: Ethernet, up to two VLAN tags, IPv4 (+options), IPv6, TCP, TCP option bytes (stack of 40), UDP; `packet_in` (offset, fromClient, msgStart, msgType, connId) and `packet_out` (kind, index); the metadata structs of each walk (`tls_meta_t`, `http_meta_t`, `dhcp_meta_t`, `x509_meta_t`, TCP options); `hs_digest_t`; the capacities; `CONN_SLOTS` = 65,536, `COPY_SESSION` = 100, `CPU_PORT` = 255.
Logic	metadata fields that must survive a resubmit or a clone are annotated `@field_list(0)` (arrival port, pending handshake record) or `@field_list(0,1)` (the copy's offset, direction, start, type, and the connection's number).
Feeds	every other P4 file (included); P4Info names used by the controller.
Why	one place for every layout: a digest struct here and the controller's field list must agree in order.

`p4/parser.p4` (237 lines)

Role	`MyParser`: Ethernet, up to two VLAN tags (802.1Q; 802.1ad outside 802.1Q), IPv4/IPv6, TCP and its options, UDP; includes the four payload walks.
Inputs	the packet; `instance_type` and `ingress_port` (resubmit → Ethernet only; port 255 → `packet_out` first).
Outputs →	headers and metadata to ingress: addresses (as 128 bits for both versions), TTL, payload length, option kinds (shifted into a 320-bit field), MSS, window scale.
Logic	option walk: `left` = data offset × 4 − 20; each byte extracted into the stack (so the deparser re-emits it), values read with `lookahead` before extraction; EOL and NOP one byte, the walk continues after EOL; malformed length → the rest kept as padding (Fig. 7). Only the first IPv4 fragment is read. VLAN tags are extracted and re-emitted by the deparser; a third tag: forwarded, not fingerprinted.
Why	JA4T/JA4TS need every option kind in order; MSS and window scale values.
Checked by	JA4T 62/62, JA4TS 51/51 exact; `test_vlan_and_ssh_port` (one and two tags).

`p4/parser_tls.p4` (356 lines)

Role	ClientHello / ServerHello walk for JA4 / JA4S; also decides “incomplete” and short segments.
Inputs	TCP payload; `payLen`; packet-out kind (3 → certificate walk).
Outputs →	`meta.tls`: hsType, versions, SNI flag, cipher(s), extension list, signature algorithms, first ALPN name (length, first and last byte), overflow bits, `walked` / `parsed` / `incomplete` → Stream, Tls, main (resubmit).
Logic	9-byte peek (record + handshake header); complete if hsLen + 9 ≤ payLen and hsLen + 4 ≤ recLen (the record may continue: ServerHello and Certificate often share one record); each list read value by value with `lookahead`, skipped with `advance`; GREASE skipped in the `select`; past the capacity: overflow bit. Segments of 1–8 bytes starting with 0x16 or a method’s first letter are treated as possible message starts. Its first state also notes whether the payload begins with “SSH-” (`meta.sshBanner`, for Ssh).
Why	every JA4 / JA4S input comes from these lists.
Alternatives	see D2, D11, D12.
Checked by	JA4 152/152, JA4S 92 + 1 explained; split/reorder 202/202; capacity test.

`p4/parser_http.p4` (395 lines)

Role	HTTP/1.x request walk for JA4H.
Inputs	a payload starting with one of nine methods (4-byte check).
Outputs →	`meta.http`: method bytes, version (last 8 bytes of the request line), header names chain (512 B), header count, Cookie and Referer flags, cookie pairs chain (1,024 B), Accept-Language (4 chars), flags → Http, Stream.
Logic	one byte per state (`HTTP_READ_BYTE`: lookahead, advance, left−1); names appended as read, with a lower-cased copy of the last 16 bytes for matching (bit 5 set); Cookie / Referer names removed afterwards by constant 6- / 7-byte slices; cookie value split at “; ” with a zero byte between pairs; bare LF accepted; payload ends before the empty line → `incomplete`.
Why	JA4H needs names in order, the cookie names and pairs, and the language.
Checked by	JA4H 72/72; split/reorder (HTTP cases); capacity test (names, cookies).

`p4/parser_dhcp.p4` (354 lines)

Role	DHCPv4 (67/68) and DHCPv6 (546/547) option walks for JA4D / JA4D6.
Logic	v4: skip the 240-byte BOOTP header and magic cookie; per option: code joins the list except 0, 50, 53, 81, 255; 53 type, 57 max size, 50 requested IP, 81 domain if the name is non-empty, 55 request list. v6: type, 3-byte transaction id, then 2+2-byte options; containers (IA_NA 3, IA_PD 25: 12 bytes; IAADDR 5: 24; IAPREFIX 26: 25; IA_TA 4: 4 and the `i` flag) skip only their fixed fields so nested options come next in packet order; 1 gives the DUID length; 39 sets `d` only for a non-empty name (fixed after the live run); 6 the requested codes.
Outputs →	`meta.dhcp` → Dhcp; `walked` → resubmit.
Checked by	JA4D/D6 10/10 on FoxIO's samples; live run; `test_dhcp6_fqdn`, capacity test.

`p4/parser_x509.p4` (371 lines)

Role	DER walk of one certificate (packet-out kind 3) for JA4X.
Logic	one shared `tlv` reader (tag, short or long-form length) entered with a return code `ret` (`R_CERT` … `R_EXT`); countdowns per level (tbs, name, set, item, extensions) mark where each SEQUENCE / SET ends; each OID's content copied as ⟨len⟩⟨bytes⟩ into the issuer, subject or extension list (256 B each).
Outputs →	`meta.x509` → X509.
Checked by	JA4X 26/26; live run (chain of 2).

`p4/main.p4` (213 lines)

Role	ingress, egress, deparser; ConnKey; ConnId (the connection's number, D21); the `forward` table.
Logic	Fig. 5 and the ingress flowchart: arrival port kept across resubmit; packets from the controller dropped after reading; controls in a fixed order; resubmit after a walk; the handshake digest from one call, postponed to pass 2 when this pass sent another digest (a message's, or Ssh's); egress E2E clone when `copy`, `packet_in` filled on the clone (with the connection's number); deparser emits `packet_in`, Ethernet, VLAN tags, IP, TCP (+options), UDP. ConnId: a new number from a counter when the slot's check differs or a SYN brings a new initial sequence number; packets from the controller keep the slot's number.
Why	the glue; every ordering rule here answers a target constraint (D5, D7, D9).
Checked by	forwarding 9,013/9,013 frames unchanged; every fingerprint test; `test_port_reuse`.

`p4/handshake.p4` (127 lines)

Role	JA4T, JA4TS, JA4L: state machine per slot (EMPTY, SYN, SYNACK, DONE, SYNACK_ONLY).
Logic	first SYN: store time A, client TTL, report JA4T; first SYN-ACK after it: time B, server TTL, JA4TS; the first ACK without SYN after that: latencyC = (C−B)/2, latencyS = (B−A)/2, JA4L; a SYN-ACK without its SYN: JA4TS only. The record goes to `meta.hsd` (sent by main.p4).
Checked by	JA4T/TS exact; TTL 102/102; `test_ja4l_gaps`.

`p4/stream.p4` (209 lines)

Role	which segments the controller must see (copy rules (a)–(e), Fig. 15).
State	per slot: connection number, client side, ISNs, started, pending, pendingSeq, edge, gap start, gap open, HTTP seen (per direction).
Outputs →	`meta.copy`, offset, direction, msgStart, msgType → egress clone; `fromClient`.
Logic	offset = seq − ISN − 1; window 16 KB; a returned joined message (or the certificates) clears pending; a retransmission bringing the whole message also clears it. Rule (e): the edge follows the furthest byte; a segment beyond it opens a gap (in a direction that carried an HTTP request, copied within 16 KB); a segment covering the gap's first byte closes it. Sequence numbers are compared modulo 2^{32} (signed difference).
Why	the switch must copy every piece a split message may need, and nothing else.
Checked by	202/202 split/reorder cases; 8 limit cases never wrong; `test_keepalive_reorder` 18/18; `test_server_flight_reorder` 19/19.

`p4/tls.p4, http.p4, ssh.p4, dhcp.p4, x509.p4`

Role	one control per fingerprint family: decide when to report, keep the per-connection state for that, send one digest (Section 3).
Logic	Tls: once per direction (`tlsDone` bits, retransmissions not twice). Http: one per request, not twice for one sequence number; the request's sequence number in the digest. Ssh: a connection on port 22 or with an "SSH-" banner (bare ACKs counted from the start); counts, batches of 8, window of 200, flush; on a reused port's SYN the previous connection's last window. Dhcp: every message. X509: every certificate with its index. DHCP, HTTP and SSH digests carry a serial number (D10); every TCP digest the connection's number (D21).

8.2 Controller (Python)

`controller/p4runtime_client.py` (201 lines)

Role	a minimal P4Runtime client: arbitration (become primary), `SetForwardingPipelineConfig`, table writes, clone session, digest configuration and acknowledgement, packet-in callback, packet-out with metadata.
Logic	a background thread reads the stream; each digest list goes to its handler, then is acknowledged; P4Info is used to translate names to ids.
Why	the only path between the controller and the switch.
Alternatives	`p4runtime-shell`: interactive, heavier; this client does only what is needed and is easy to read.

`controller/main.py` (240 lines)

Role	the Controller: start-up (program, forwarding, clone session, six digests), one handler per digest, packet-in to the reassembler and packet-out back, `finish()` (SSH flush).
Outputs →	records (`records.py`), console lines, JSON file.
Logic	each handler decodes its struct (field order = the P4 struct), finds the connection's record by its endpoints and number, sets the client / server roles (the SYN sender, the ClientHello sender, the SSH digest's direction), writes the text via the formatting module. JA4H are kept in stream order by each request's sequence number. An SSH connection is remembered for the final flush; an earlier one on the same endpoints is not flushed (its window came on the new SYN).

`controller/handshake.py`, `tls.py`, `http.py`, `ssh.py`, `dhcp.py`, `x509.py`

Role	decode one digest struct and write FoxIO's text; no packet bytes.
Logic	handshake: `window_kinds_mss_scale`, JA4L halves. tls: version code (supported_versions if present), SNI flag, two-digit counts (capped at 99 as FoxIO), ALPN first / last characters ('9' for a non-ASCII byte, '0' for a missing one, `00` without ALPN), sorted ciphers; sorted extensions without SNI and ALPN; signature algorithms in order; SHA-256 first 12 hex digits. http: method code, version, flags, count, language; three hashes. ssh: histogram and mode per direction (smallest length on a tie, as FoxIO). dhcp: type names, size, flags, lists. x509: OIDs as hex joined by ','.
Checked by	every EXACT row of the comparison.

`controller/reassembly.py` (199 lines)

Role	join split messages by framing (Section 3.2 and the algorithm in Section 9).
Inputs	copied segments with (endpoints, connection number, direction), seq, offset, start flag and type.
Outputs →	complete messages with the sequence number of their first byte → packet-out kind 1, or a list of certificates → packet-out kind 3 each.
Logic	bounded (D24): 32 KB per direction (lowest offsets dropped first), idle directions dropped after 60 s, at most 100,000 directions; a running byte count, so the bound costs nothing per piece.
Checked by	split/reorder 202/202; `test_controller_state` (bounds, a false start inside a request).

`controller/records.py` (57 lines)

Role	one record per connection (key: endpoints in a fixed order and the switch's connection number), first value of each name kept; lists for JA4H, JA4SSH, JA4D; per-message list for DHCP in arrival order. Records are kept until the run ends and written to JSON (a long-running deployment would export and drop them; see the assumptions).

8.3 Tools, tests, network

`tools/replay.py` (242 lines) and `tools/nanoport.py` (57 lines)

Role	run the real switch and controller without root and send a capture through them.
Logic	free gRPC / Thrift ports; ipc path in the runtime folder; switch dies with its parent; one nanomsg socket, opened after the switch answers; warm-up frame; client / server side by the SYN sender; gaps kept (max 2 s); `sent.pcap` with send times; frames out collected; forwarding count.
Outputs →	records, messages, `sent.pcap`, forwarding numbers → `compare.py` and the tests.

`tools/reference.py` (172 lines)

Role	the references: FoxIO's Rust tool (JSON per connection), tshark fields for JA4TS, FoxIO's stored Wireshark plugin output for JA4D, and the plugin's rule on tshark's PDML for captures without stored output (skipping DHCP quoted inside ICMP errors). A later TCP connection on the same endpoints is keyed with its place among them, from tshark's stream numbers (which FoxIO's tool reports).

`tools/compare.py` (219 lines)

Role	per capture: replay, then a verdict per fingerprint per connection; totals; forwarding; JA4L distribution; not-compared counts (HTTP/2, TLS 1.3 certificates). `compare_records` is shared with the live run.
Outputs →	console report, `results/compare_all.json` → the results tables of both PDFs.
Logic	EXPLAINED only for differences checked by hand and written with their reason; EXCLUDED only tunnel captures; exit code non-zero on any WRONG or any altered frame. The switch's records are keyed like the references: endpoints, and the place among connections on them in the order of the connection numbers.

`tools/measure_field_counts.py` (326 lines)

Role	the measurement study of list lengths (capacities).
Logic	local listener records the first message of each installed client; corpus of 42 captures; tshark PDML (reassembled) for TLS / HTTP, the JA4D rule for DHCP, `cryptography` for certificates; per list: p50, p95, p99, max, distinct, share within the capacity.
Outputs →	`results/measure/field_counts.json` → `results_field_counts.tex`.

`network/run_live.py` (249 lines)

Role	the live run: Mininet h1 – s1 – h2 with BMv2 on real interfaces, real servers and clients, captures on the switch ports, comparison and forwarding check.
Logic	offloads off; certificates made for a chain of 2; servers' process ids kept; clients start once h2 listens on every server port (`ss`); every client under `timeout -k 2 15`, its exit status checked; a Ctrl-C interrupts the running host command before cleanup; results chowned to the user.
Outputs →	`results/live/` (report, rows, records, captures) → `tools/live_tex.py`.

`tools/live_tex.py` (45 lines)

Role	writes the live run's results table for both PDFs, so the numbers are never typed by hand.
Logic	counts verdicts per fingerprint in `rows.json`; forwarding lines from `report.txt`; JA4L latency differences recomputed from the rows (median as the live run prints it, the upper middle value).
Outputs →	`docs/results_live.tex` → `design.pdf` and this document (`make live` runs it).

`tests/` (11 files, 27 tests) and `Makefile`

Role	tests in Section 5; `make build / compare / test / live / docs / clean`.

9 The algorithms, step by step

Pseudo-code that follows the code line for line (names as in the files). P4 parser states are written as labelled steps; "read" means `lookahead` then `advance` (or `extract` where the bytes must be re-emitted).

9.1 Connection key (`main.p4`, ConnKey)

```
(a1,p1,a2,p2) := (src,sport,dst,dport) if (src,sport) <= (dst,dport) else (dst,dport,src,sport)
srcIsFirst    := whether the source came first
slot  := CRC16(ipVersion, a1, p1, a2, p2) mod 65536        -- index of every per-connection register
check := CRC32(ipVersion, a1, p1, a2, p2)                  -- stored in the slot; a mismatch = another connection
```

Why: the same slot from both directions; two independent hashes so that a slot collision is detected.

9.2 Connection number (`main.p4`, ConnId)

```
read connCheck[slot], connNumber[slot] (id), connSynSeq[slot]
another := connCheck != check or id == 0
reused  := not another and packet is a SYN (not SYN-ACK) and seq != connSynSeq
real packet:
   if another or reused:  id := ++counter; connCheck := check; connNumber := id
                          if reused: connReused := 1; prevConnId := the old id
   if SYN: connSynSeq := seq
   connId := id
from the controller:  connId := (another ? 0 : id)          -- never a new number
```

Why: a repeated SYN (same sequence number) is the same connection; a SYN with a new one on the same endpoints is port reuse. Every control compares its stored number with `connId` and starts over when they differ.

9.3 TLS hello walk (`parser_tls.p4`)

```
peek 9 bytes: type, version, recLen | hsType, hsLen
if type != 0x16 or version major != 3 or hsType not in {1,2}:  try HTTP
if not (hsLen + 9 <= payLen and hsLen + 4 <= recLen):          incomplete := 1; stop   -- Stream copies it
walked := 1; skip 9; legacyVersion := read16; skip random (32); skip session id (1 + n)
ClientHello: n := read16 (cipher bytes)
   repeat n/2 times: v := read16
       if GREASE(v): continue
       if cipherCount == 128: overflow |= 1; continue
       ciphers := ciphers << 16 | v; cipherCount += 1
   skip compression methods
ServerHello: cipher := read16; skip compression (1)
extensions: left := read16
   while left > 0: type := read16; len := read16
       ClientHello and GREASE(type): skip body; continue
       append type (cap 64, overflow bit 2)
       type 0  (SNI):                 sni := 1
       type 16 (ALPN):                first name: alpnLen, first byte, last byte
       type 43 (supported_versions):  maxVersion := highest non-GREASE (ServerHello: the selected one)
       type 13 (signature_algorithms):each non-GREASE value appended (cap 64, overflow bit 4)
       skip what is left of the body
parsed := 1
GREASE(v) := (v & 0x0f0f) == 0x0a0a and high byte == low byte
```

9.4 HTTP request walk (`parser_http.p4`)

```
first 4 bytes must be one of: "GET " "POST" "PUT " "HEAD" "DELE" "OPTI" "CONN" "PATC" "TRAC"
method  := bytes up to the first space (at most 8 kept)
skip the target up to the next space
version := the last 8 bytes before the line end                 -- e.g. "HTTP/1.1"
loop over lines:
   empty line (CR LF or LF)        -> parsed := 1; stop
   name bytes up to ':'            -> appended to names (one byte at a time, chain of 2 x 2048 bits),
                                      lower-cased copy of the last 16 bytes in cur
   cur == "cookie"                 -> hasCookie; remove the 6 name bytes again (constant slice);
                                      first Cookie value: pairs split at "; ", zero byte between pairs
   cur == "referer"                -> hasReferer; remove the 7 name bytes again
   any other name                  -> headerCount += 1, a zero byte after it in names
   cur == "accept-language"        -> (also counted) first value: up to 4 characters before ',', '-' dropped
   other values                    -> skipped to the line end
   payload exhausted before the empty line -> incomplete := 1    -- Stream copies it
```

Lower-casing sets bit 5 of every byte: for letters it gives the lower case, and ’-’ (0x2D) already has bit 5 set, so the three names compare correctly whatever their case.

9.5 DER walk with return codes (`parser_x509.p4`)

```
ret := R_CERT; goto tlv
tlv:    tag := read8; b := read8
        if b < 0x80: len := b  else: len := next (b & 0x7f) bytes, big-endian
        select ret:
          R_CERT    -> ret := R_TBS;     goto tlv                  -- Certificate SEQUENCE: go inside
          R_TBS     -> tbsLeft := len;   ret := R_VERSION; goto tlv
          R_VERSION -> tag 0xA0? skip; ret := R_SERIAL ... (serial, signature: skipped)
          R_NAME    -> nameLeft := len   -- issuer first, subject second
          R_SET / R_ATV -> setLeft, itemLeft countdowns
          R_OID     -> copy <len><content bytes> to issuer | subject | exts (256 bytes each, overflow bit)
          R_VALIDITY, R_SPKI -> skipped
          R_TBS_OPT -> [1] [2] skipped; [3] -> R_EXTSEQ -> each extension: R_EXT -> its OID
every element read decreases the countdown of every level that contains it; a countdown at 0 ends that level
```

9.6 Handshake and JA4L (`handshake.p4`)

```
now := ingress_global_timestamp (us)
SYN only:   if new connection or state in {DONE, SYNACK_ONLY}: state := SYN; A := now; ttlC := ttl; report JA4T
SYN+ACK:    if state == SYN: state := SYNACK; B := now; ttlS := ttl; report JA4TS
            if another connection held the slot: state := SYNACK_ONLY; report JA4TS
ACK only:   if state == SYNACK: latencyC := (now - B) >> 1; latencyS := (B - A) >> 1;
                                state := DONE; report JA4L
the record is put in meta.hsd; main.p4 sends it from one digest call (on pass 2 if a message digest was sent)
```

9.7 Which segments are copied (`stream.p4`)

```
SYN from the client:  ISN_client := seq; started := 0; pending := 0
SYN-ACK:              ISN_server := seq
no ISN and a message starts here: ISN := seq - 1           -- the stream is taken to start with this message
offset := seq - ISN - 1
(a) message starts here and is incomplete      -> pending; pendingSeq := seq; copy (start, TLS/HTTP)
(d) complete TLS <= 1.2 ServerHello            -> pending; copy (start, type 3: certificates follow)
(b) pending and seq - pendingSeq < 16384       -> copy
(c) direction not started, offset in 1..16383  -> copy    -- a piece that may precede the first segment
(e) HTTP seen in this direction and aheadOfGap  -> copy    -- a later request's piece before its first segment
gap bookkeeping (started directions; differences of sequence numbers taken as signed 32-bit):
   gap open: seq <= gap < seq+len -> gap closed;  0 < seq-gap < 16384 -> aheadOfGap;  seq-gap >= 16384 -> given up
   no gap:   seq > edge -> gap := edge; open; aheadOfGap
   edge := max(edge, seq + len)                  -- edge starts at ISN + 1
offset == 0 -> started
a joined message (or certificate) returned at pendingSeq -> pending cleared
```

9.8 Joining (`reassembly.py`)

```
add(key, seq, offset, start, type, payload):          -- key: (endpoints, connId, direction)
   once a second: drop directions idle for 60 s; above 100,000 directions: drop the least recently used
   base := seq - offset
   pieces[offset] := (bytes already held at offset) + (the rest of payload)   -- first received wins
   if start: starts[offset] := type
   for each start not done, in order:
       data := contiguous(pieces, start)        -- up to the first gap, at most 16384 + 9 bytes
       TLS:    strip 5-byte record headers, join fragments until 4 + hsLen bytes -> one record
       HTTP:   data up to the first \r?\n\r?\n
       flight: handshake messages until type 11 -> its certificates (3-byte length each); type 14 first -> []
       complete -> done; return (base + start, type, message)
   more than 32 KB held: drop pieces from the lowest offset up (and starts below the lowest piece)
```

9.9 JA4SSH (`ssh.p4`, `ssh.py`)

```
switch, per segment of an SSH connection (port 22, or an "SSH-" banner seen; direction: port 22 side = server,
   else the SYN sender = client):
   reused port's SYN and the old connection had counts: FINAL digest for the old connId; reset
   not SSH yet: only bare ACKs are counted (FoxIO counts the handshake's ACK of a connection found by its banner)
   payload > 0: count SSH packet per direction; batch := batch << 16 | (direction bit | length)
   no payload and flags == ACK: count bare ACK per direction
   sshClient + sshServer == 200: WINDOW_END digest (counts + last lengths); reset
   batch holds 8: BATCH_FULL digest; batch reset
   flush packet: FINAL digest (current counts and lengths); reset
   first banner on a port other than 22: START digest (the controller remembers the connection for the flush)
controller: lengths of the window per direction in a Counter;
   mode := min{ length : count == max count }; JA4SSH := c<modeC>s<modeS>_c<sshC>s<sshS>_c<ackC>s<ackS>
```

9.10 Text and hashes (controller)

```
hash12(s)  := first 12 hex digits of SHA-256(s), "000000000000" for an empty s
JA4  := t + version + (d|i) + %02d ciphers + %02d extensions + alpn
        _ hash12(sorted ciphers as %04x, ',')
        _ hash12(sorted extensions without 0000 and 0010, ',' + '_' + signature algorithms in order)
JA4S := t + version + %02d extensions + alpn _ %04x cipher _ hash12(extensions in order)
JA4H := method + version + (c|n) + (r|n) + %02d count + lang(4) _ hash12(names) _ hash12(sorted cookie names)
        _ hash12(sorted cookie pairs)
JA4X := hash12(issuer OIDs hex, ',') _ hash12(subject ...) _ hash12(extensions ...)
```

9.11 Verdicts (`compare.py`)

```
for each connection key in switch | reference, for each fingerprint name:
   neither has it           -> nothing
   only reference           -> MISSING      only switch -> EXTRA
   JA4L                     -> EXACT if the TTLs are equal (latency difference kept as a measurement)
   otherwise                -> EXACT if equal else WRONG
   listed in EXPLAINED (checked by hand, with the reason) and not EXACT -> EXPLAINED
DHCP: the sequence of messages compared position by position
run fails if any WRONG or any frame not forwarded unchanged
```

10 Corner cases and how each is handled

Case	What happens	Where	Checked by
Messages over several segments			
hello / request split, in order	first segment: incomplete → copy (a); later pieces (b); joined, sent back, read	stream, reassembly	split/reorder
a later piece arrives first (SYN seen)	copied by (c): the direction has not started; placed by its offset when the start arrives	stream (c)	split/reorder
a later piece first, no SYN seen	cannot be placed (offsets unknown): the message is not reported, never wrong	documented limit	8 limit cases
segments duplicated / retransmitted	bytes already held are kept; the hello is reported once per direction (`tlsDone`); a request not twice for one sequence number	reassembly, tls, http	split/reorder
overlapping segments with different bytes	the bytes received first are kept	reassembly	split/reorder
hello spread over several TLS records	record headers removed, fragments joined, one record sent back	reassembly	split/reorder

Case	What happens	Where	Checked by
ServerHello and Certificate in one record	only the hello must be complete	parser_tls	comparison
first segment of 1–8 bytes	treated as a possible start if it begins with 0x16 or a method's first letter	parser_tls	split/reorder
message after other bytes (SOCKS, CONNECT)	read when whole in a segment; a TLS hello there split and out of order: limit (rule (e) covers HTTP directions only)	stream	comparison
a later request on a keep-alive connection, split, second piece first	pieces ahead of the gap copied by (e); joined when the first piece arrives	stream (e)	keep-alive test
a joined request reported after later whole ones	the connection's JA4H listed by each request's sequence number	http, main.py	keep-alive test
a false start inside a request (a 1-byte piece beginning like a method)	"joined" first and ignored by the switch; the real request is still joined from the same bytes	reassembly	state test, split/reorder
sequence numbers wrapping	offsets in 32-bit arithmetic	stream, reassembly	by construction
Digests			
two fingerprints on one packet (hello on the handshake ACK)	the handshake digest waits for the second pass	main	one-digest test
identical messages close together	serial number in DHCP, HTTP, SSH digests	dhcp, http, ssh	identical-digest test
TLS			
GREASE in ciphers, extensions, signature algorithms, versions	skipped in the parser; ServerHello extensions kept as they are (as FoxIO)	parser_tls	comparison
lists past the capacity	count stays right, overflow bit, "incomplete"	parsers, controller	capacity test
non-ASCII ALPN	'9' for the character, as FoxIO	tls.py	comparison
TLS 1.3	certificates encrypted: no JA4X (not compared, counted)	–	comparison
TLS 1.2 chain over many segments	flight copied (d)+(b), certificates cut and sent back one by one	stream, reassembly	live run, flight test
server's later segment first (certificates before the ServerHello)	copied by (c); the ServerHello's segment starts the flight (d); joined when the gap closes	stream, reassembly	flight test
TLS 1.2 ServerHello itself split	JA4S joined as any hello; no certificate copy starts: JA4X not reported (limit; a ServerHello is about 100 bytes, so TCP rarely splits it)	stream	flight test
flight without Certificate	ServerHelloDone first: nothing to report	reassembly	by construction
TCP and IP			
padding after EOL	each padding byte listed as another 0, as FoxIO	parser	comparison
malformed option length	rest of the options kept as padding, not read	parser	by construction
IPv4 options	parsed (varbit) and re-emitted	parser	forwarding check
IPv4 fragments	only the first fragment is read beyond IP	parser	by construction

Case	What happens	Where	Checked by
IPv6 extension headers	not walked: forwarded, not fingerprinted	limit	–
SYN-ACK without its SYN	JA4TS only; no JA4L	handshake	comparison
port reused: a new SYN with a new sequence number on the same endpoints	a new connection number: every control starts over, the controller opens a new record; an SSH connection's last window is reported on that SYN	ConnId, ssh, main.py	port-reuse test
the same SYN repeated	same number: nothing restarts	ConnId, handshake	port-reuse test
one or two VLAN tags (802.1Q; 802.1ad outside)	parsed and re-emitted; fingerprinted as untagged	parser	VLAN test
three or more VLAN tags	forwarded, not fingerprinted	limit	–
two connections in one slot	told apart by the 32-bit check; the newer takes the slot	ConnKey	by construction
HTTP			
bare LF line ends	accepted	parser_http	comparison
names in any case	compared lower-cased (bit 5), listed as sent	parser_http	comparison
several Cookie headers	the first one's pairs are used	parser_http	not tested
several requests per connection	one JA4H each, in order	http	comparison
unknown method or version	no JA4H (FoxIO computes none)	http.py	comparison
HTTP/2, HTTP/3	inside TLS / QUIC: not compared, counted	–	comparison
SSH, DHCP			
last, partial SSH window	flush packet at the end	ssh, main.py	comparison
SSH on a port other than 22	recognised by its "SSH-" banner; bare ACKs before it counted (as FoxIO); the SYN sender is the client	ssh, parser_tls	SSH-port test
malformed / SSH v1 streams	every port-22 payload segment counted (tshark refuses a few): EXPLAINED	ssh	comparison
DHCP retransmissions	every message reported	dhcp	live run
DHCPv6 nested options	containers skip only their fixed fields	parser_dhcp	comparison
DHCPv6 empty FQDN name	flag `n`	parser_dhcp	dhcp6 test
DHCP quoted in an ICMP error	not a DHCP message: not parsed (switch), skipped (reference)	reference	live run
Switch and controller			
controller buffers on a long run	32 KB per direction, idle 60 s, 100,000 directions	reassembly	state test
packets from the controller	read, never forwarded	main	forwarding check
walked packets	resubmitted, original bytes forwarded	main	forwarding check

11 Problems found and how they were fixed

Shared with `design.pdf` (one file, `docs/problems.tex`): every error and corner case met while building the project. Where a test was written for a bug, it failed on the old program before the fix and passed after it.

What went wrong	Found by	Cause and fix	Checked by
Reading packets (P4 parser)			
MSS 1460 read as 46260	JA4T vs FoxIO, first comparison	the value was read back through the header stack's `.last` after extraction, which gave the wrong bytes. Values are now read with `lookahead` before the bytes are extracted	comparison (JA4T 62/62)
TCP options `-0-0` vs `-0`	JA4T vs FoxIO	FoxIO lists every padding byte after End-of-Options as another option 0. The walk stopped at EOL; it now treats EOL as a one-byte option and continues	comparison
two ServerHellos missing	comparison, `latest.pcapng`	a hello was required to fill its whole TLS record; servers put ServerHello and Certificate in one long record. Now only the hello itself must be complete (hsLen $+ 9 \leq$ payLen and hsLen $+ 4 \leq$ recLen)	comparison
SOCKS / CONNECT hellos missing	comparison (a regression)	the state "this stream does not start with a hello" also stopped the hello from being reported. It now only stops early pieces from being copied	comparison (`socks*`, `https-connect`)
a split inside the first 9 bytes not joined	split test: 85 of 112 cases	a first segment shorter than the TLS headers cannot show that a hello starts there. A short segment starting with `0x16` (or a method's first letter, for HTTP) now counts as a possible message start and is copied	`test_split_reorder` 202/202
HTTP request with bare LF line ends	comparison	only CR LF ended a line; FoxIO accepts LF alone. The walk now does too	comparison (JA4H 72/72)
JA4D6 flag `d` for an empty name	live run	`udhcpc6` sends option 39 with the flags and a single `0x00` (the root, an empty name); "length > 1" counted it as a name, FoxIO does not. A name is now present when longer than one byte, or one byte not zero	`test_dhcp6_fqdn`; live rerun exact
lists longer than the switch keeps	measurement study (a path no test exercised)	not a bug: the overflow path had no test. A message one past each capacity is now sent and must be reported incomplete; one at the capacity must equal FoxIO	`test_capacity_limits`
VLAN-tagged frames not fingerprinted	review of assumptions; FoxIO's tool reads through tags	the parser went from Ethernet straight to IPv4 / IPv6, so a tagged frame was forwarded with nothing read. Up to two tags are now parsed (802.1Q; 802.1ad outside 802.1Q) and re-emitted	`test_vlan_and_ssh_port`: red (6 MISSING), then exact
SSH on a port other than 22 not fingerprinted	review of assumptions; FoxIO's tool gave JA4SSH on port 2222	the switch counted port 22 only; tshark recognises SSH by its banner on any port. The parser notes a payload beginning with "SSH-"; that marks the connection. The first version gave `c43s43_c7s6_c6s6` for FoxIO's `…_c7s6`: the handshake's ACK comes before the banner and FoxIO counts it. Bare ACKs are now counted from the start of every connection	same test: exact

What went wrong	Found by	Cause and fix	Checked by
Compiler and target limits			
fields over 2,048 bits; variable shifts	p4c / BMv2 errors	BMv2 fields are at most 2,048 bits, variable shifts at most 8 bits, constant ones 256. Long lists became chains of 2,048-bit fields, grown by constant-width slices	build
a table applied in two places	p4c error	`forward` was applied in each branch; it is now applied once, at the end of ingress	build
two parser states of the same name	p4c error	the TLS and certificate walks both had `ext_next`; the certificate states became `x_ext_next`, `x_ext_read`	build
Switch state and digests			
JA4L digests before JA4T	digest order in the controller	p4c makes one learn list per `digest()` call site, and the controller's settings reached one of them. Every control now has a single call	comparison
JA4L missing on SSH connections	comparison	BMv2 keeps one digest per packet; the handshake ACK sent JA4L and an SSH start record. The start record was removed and the handshake digest is sent once, after any message digest of the same packet	`test_one_digest_per_packet` (red, then green)
identical messages close together reported once	replay of the live capture	BMv2 drops a digest identical to one not yet acknowledged (Section 7.10). DHCP, HTTP and SSH digests now carry a serial number	`test_identical_digests`: DHCP 3/5 and HTTP 2/6 runs failed before; SSH (with acknowledgements held back) 3/4; all pass after; live rerun: 3+3 identical DHCP messages all reported
registers not readable over P4Runtime	BMv2 answers UNIMPLEMENTED	the last, partial SSH window was to be read from registers. The controller now sends a flush packet; the switch reports the window itself	comparison (JA4SSH)
hello packets not forwarded	byte-for-byte frame comparison	the second pass (resubmit) lost the arrival port, so `forward` missed and the packet was dropped. The port now travels in metadata kept across the resubmit (field list 0)	forwarding check on every run (9,013/9,013)
copies without their payload	joined messages empty	an ingress copy is parsed again from the start, so the walk consumed its bytes again. Copies are now made at egress (E2E), where the packet is whole	`test_split_reorder`

What went wrong	Found by	Cause and fix	Checked by
a new connection on reused ports mixed with the old one	review of assumptions; `test_port_reuse`	the controller keyed records by endpoints and kept the first value, so the second connection's JA4 was dropped; the SSH counts of two sessions were added together into a wrong JA4SSH (`c14s12` for `c7s6`). Control ConnId now numbers connections (new number on another check or on a SYN with a new sequence number); every control keeps state with it; every TCP digest and copy carries it; the controller keys by it; the previous SSH connection's last window is reported on the new SYN (its handshake digest then goes on a second pass)	`test_port_reuse`: red, then every fingerprint of both connections exact
later request on a keep-alive connection lost when split and out of order	review of assumptions; `test_keepalive_reorder`	rule (c) copies early pieces only before a direction has started. Rule (e): the switch keeps each direction's edge and one gap; in a direction that carried an HTTP request, segments ahead of the gap (within 16 KB) are copied until it closes	`test_keepalive_reorder` 18/18
JA4H of a connection out of order	the same test: every case failed, in order too	a joined request comes back from the controller after the whole requests that followed it; FoxIO lists them in stream order. The HTTP digest now carries the request's sequence number and the controller keeps the list in that order	same test
JA4L (timing)			
latency differences of a few ms	comparison with original timing	BMv2 stamps a packet when its pipeline takes it up, not when it arrives; a packet behind a slow walk is stamped late. JA4L is reported as a measurement: TTL scored, latency as a distribution	`test_ja4l_gaps` (arithmetic)
gap test failing now and then	repeated test runs	short gaps were within the 2-CPU VM's scheduling noise; the gaps are now 1 s and 0.5 s, checked within 2%	`test_ja4l_gaps`
gap test failing under heavy load	`make test` run beside the measurement study (headless Firefox)	BMv2 took the SYN into its pipeline 130 ms after it was handed over, so the server half was 13% short; the switch's stamp is the pipeline's, not the arrival's (Section 5). Not changed: the test checks arithmetic and needs an idle machine; a wider tolerance would hide a wrong halving. Passes 3/3 when idle; 6 of 8 with a browser open on the 2-CPU machine (2026-10-12)	`test_ja4l_gaps`
Reference and comparison			
loopback captures not read	comparison	FoxIO's loopback captures have no Ethernet header; one is added before replay (IPv4 / IPv6 decoded by hand)	comparison
DHCP inside ICMP errors counted	live run	tshark dissects the DHCP message quoted inside "port unreachable" as DHCP; the reference counted four extra messages. It now skips ICMP	equal to FoxIO's plugin output on both samples; live rerun: 10 of 10 DHCP messages exact

What went wrong	Found by	Cause and fix	Checked by
reused ports invisible to the comparison	building `test_port_reuse`	the references were keyed by endpoints, so a second connection overwrote the first. FoxIO's tool reports tshark's stream number: a later connection on the same endpoints is now keyed with its place among them, the switch's records in the order of their connection numbers	same test (4 JA4T rows, 2 JA4, 2 JA4SSH compared)
Test bench and live run			
a stale switch answering a new run	results of an old program	a switch from a stopped run held the fixed gRPC port. Each run takes free ports; the switch dies with its parent (`PR_SET_PDEATHSIG`)	every run
packet socket errors	bench start-up failures	a nanomsg socket was opened per send and leaked; a Unix socket path is limited to 107 characters. One socket per run, closed at the end, with a unique short path in the runtime folder	every run
live run hung on the 2 KB ClientHello	live run	`openssl s_client -quiet` ignores the end of its input and waited for the time limit. `-quiet` removed; `timeout -k 2 15` on every client	live rerun completed, every step 0.1–3.3 s
Ctrl-C left the network half up	live run	Mininet cannot clean up while a host runs a command; the script now interrupts that command first. `dnsmasq` and `sshd` were not stopped: every server's process id is now kept	live rerun completed, every step 0.1–3.3 s
new flight test: one case failed	first run of `test_server_flight_reorder`	a test bug, not a switch bug: in one case the certificate pieces' offsets were not shifted past the ServerHello, so the switch received wrong bytes and (correctly) reported no certificate. Traced by logging every copy the controller received; offsets fixed	19 of 19 exact
`sshd` would not start	live run preparation	`sshd` needs `/run/sshd`, made only by the system service. The script creates it	live rerun completed, every step 0.1–3.3 s
controller buffers grew without end	review of assumptions	the reassembler kept every copied piece. Now at most 32 KB per direction (lowest offsets dropped first), idle directions dropped after 60 s, at most 100,000 directions	`test_controller_state` (red: no limits; then green)
split/reorder fell to 199 of 202 while bounding the buffers	`make test`	my first version also dropped a message's bytes as soon as it was joined. A one-byte segment in the middle of a request that begins like a method ("G") was "joined" as a message first, its bytes dropped, and the real request, whose first segment came last, could not be completed. Early dropping removed; the case is now a unit test	202/202; `test_false_start_inside_a_message`

What went wrong	Found by	Cause and fix	Checked by
live run without JA4H (all rows still EXACT)	live rerun after the connection-number fix	not the switch: h2 answered the three HTTP connections with RST. The script started `python3 -m http.server` and waited a fixed 2 s; the server was not yet listening, and the reference saw the same frames, so nothing was flagged. The script now waits until h2 listens on every server port (up to 30 s) and checks each client's exit status; a failed step is printed and goes into the results table	final live run: all 9 steps as expected, JA4H exact
the live DHCP client never got a lease (every live run)	the new exit-status check: DHCP step exit 1	dnsmasq pings an address and waits about 3 s before offering it; udhcpc's three tries take 3 s, so the Offers arrived after it gave up (and h1 answered them with "port unreachable", the ICMP-quoted DHCP of an earlier row). Only Discover and Offer were ever exercised. dnsmasq now runs with `--no-ping`	final live run: Discover, Offer, Request, ACK, all exact

12 Results

The tables are the same files as in `design.pdf` (`docs/results_*.tex`), regenerated from the result files after every run.

12.1 FoxIO's sample captures, replayed

Fingerprint	EXACT	EXPLAINED	WRONG	MISSING	EXTRA
JA4T	62	0	0	0	0
JA4TS	51	0	0	0	0
JA4L-C (TTL)	51	0	0	0	0
JA4L-S (TTL)	51	0	0	0	0
JA4	152	0	0	0	0
JA4S	92	1	0	0	0
JA4H (per connection)	72	0	0	0	0
JA4SSH (per connection)	6	4	0	0	0
JA4X (per chain)	26	0	0	0	0
JA4D / JA4D6 (per message)	10	0	0	0	0
Total	573	5	0	0	0

All 33 of FoxIO's sample captures that hold traffic the switch reads, replayed with their original timing; 3 tunnel captures excluded. **Forwarding: 9,013 of 9,013 frames left the switch byte for byte unchanged.** JA4L latency (a measurement, see above), difference to the reference over the same frames: median 331 µs, 90th percentile 1341 µs, maximum 4759 µs (2 of the halves above 2 ms). Not compared: 16 JA4H of HTTP/2 requests and 3 JA4X of TLS 1.3 certificates that FoxIO's tool could compute only by decrypting with keys stored in the captures. The 5 EXPLAINED: one SOCKS4 ServerHello the reference cannot decode (checked by hand), four JA4SSH of malformed or SSH v1 streams where tshark refuses a few segments as SSH. Source: `results/compare_all.json`.

12.2 The live run

The final run, on the build with every fix: all 9 steps ended as expected (the script now waits for the servers and checks every client's exit status), every fingerprint matched FoxIO, and DHCP completed Discover, Offer, Request, ACK (dnsmasq `--no-ping`; earlier runs never got past the Offer, see the problems table). For the split ClientHello the console prints JA4S before JA4, and the 3 KB request's JA4H after the step ends: both wait for the controller's join, as designed.

Fingerprint	EXACT	EXPLAINED	WRONG	MISSING	EXTRA
JA4T	7	0	0	0	0
JA4TS	7	0	0	0	0
JA4L-C (TTL)	7	0	0	0	0
JA4L-S (TTL)	7	0	0	0	0
JA4	3	0	0	0	0
JA4S	3	0	0	0	0
JA4H (per connection)	3	0	0	0	0
JA4SSH (per connection)	1	0	0	0	0
JA4X (per chain)	1	0	0	0	0
JA4D / JA4D6 (per message)	8	0	0	0	0
Total	47	0	0	0	0

One live run of `network/run_live.py`, compared on the frames that entered the switch. **Forwarding: s1-eth1 → s1-eth2: 96 of 96; s1-eth2 → s1-eth1: 86 of 86 frames left the switch unchanged.** JA4L latency (a measurement), difference to the reference over the same frames: 14 halves, median 68 µs, maximum 707 µs. Source: `results/live/rows.json`.

12.3 Capacities: the measurement study

Quantity	cap	msgs	distinct	p50	p95	p99	max
TLS ClientHello cipher suites	128	241	132	15	28	46	86
TLS ClientHello extensions	64	241	132	16	16	18	18
TLS ClientHello signature algorithms	64	241	132	8	15	20	20
TLS ServerHello extensions	64	135	27	2	6	6	7
HTTP header-name bytes	512	135	29	51	146	146	146
HTTP cookie bytes	1,024	7	7	108	863	863	863
DHCP option codes	64	20	16	5	7	7	7
DHCP requested codes	128	20	16	0	7	7	7
certificate issuer OID bytes	256	66	47	16	20	26	26
certificate subject OID bytes	256	66	47	16	26	60	60
certificate extension OID bytes	256	66	47	37	72	72	72

1238 of 1238 measured lists (100%) are within their capacity. "distinct": messages counted once per distinct content, so one chatty client does not weigh more. 42 captures; certificates `cryptography` could not parse: 0. Source: `results/measure/field_counts.json`.

12.4 Tests

`make test`: 11 files, 27 tests, all passing on an idle machine; split / reorder 202 of 202 cases exact; the server's TLS 1.2 flight split and out of order 19 of 19 exact (JA4S and both JA4X); a later request on a keep-alive connection split and out of order 18 of 18 exact; port reuse (TLS and SSH), VLAN tags (one and two) and SSH on port 2222 exact; the controller's buffers

bounded; the 8 documented limit cases never wrong. (`test_ja4l_gaps` needs an idle machine: under heavy load BMv2 took a packet up 130 ms late once; see the problems table.)

13 What is done, what is not, and why

The claim the evidence supports. For clear-text TCP and DHCP traffic through the switch, every JA4+ fingerprint that can be computed passively without keys (JA4, JA4S, JA4H, JA4X, JA4T, JA4TS, JA4L, JA4SSH, JA4D, JA4D6) is computed in the P4 data plane and matches FoxIO's own tools, for messages whole, split, reordered, duplicated or overlapping, sent by the client or by the server. The project does not claim that fingerprinting is complete in every respect: the second table lists what is missing and why.

Done, and the evidence

Fingerprint	**FoxIO's samples** (exact / explained)	**final live run**	**tests**
JA4T, JA4TS	62, 51	7, 7	split / reorder bench; port reuse; VLAN; one digest per packet
JA4L (C, S)	TTL 51 / 51; latency as a measurement (median 331 µs off the reference)	7, 7	`test_ja4l_gaps` (1 s and 0.5 s gaps, within 2 %; needs an idle machine)
JA4	152	3 (one hello split by TCP)	202 split / reorder cases; capacity edges; port reuse; VLAN
JA4S	92 / 1	3	split / reorder (ServerHello); server flight in 19 orders
JA4H	72 (per connection)	3 (one 3 KB request split by TCP)	split / reorder (HTTP); keep-alive in 18 orders; capacity; identical requests; two VLAN tags
JA4X	26 (per chain)	1 chain of 2	server flight in 19 orders
JA4SSH	6 / 4	1	identical windows; port reuse; SSH on port 2222
JA4D, JA4D6	10	8 (Discover, Offer, Request, ACK; Solicit, Advertise, Request, Reply)	identical messages; DHCPv6 empty name; capacity
forwarding	9,013 / 9,013 frames unchanged	182 / 182	every replay test

Not done, or not proven, and why

What	Effect	Why it is not done	What doing it would take
Fingerprints not implemented			
JA4 over QUIC (`q…`), the client's HTTP/3	no JA4 for HTTP/3 browser traffic, a large share of real traffic	the scope set for the project is TCP and DHCP. QUIC carries the ClientHello in Initial packets protected with keys derived from public values (HKDF over the connection id, AES-GCM, header protection): readable passively, so FoxIO computes it, but v1model has no AES or HKDF, so the P4 pipeline cannot decrypt	the controller decrypts the Initial packets and sends the CRYPTO frames back for the same TLS parser (as joined messages are now); a scope decision for the faculty
JA4 over DTLS (`d…`)	no JA4 for DTLS (WebRTC, VPNs)	not in the scope; not in the test traffic	a DTLS record parser in front of the TLS walk (clear hellos, no decryption)
JA4TScan	–	an active fingerprint (it sends probes); a switch that observes traffic does not probe	not applicable
Traffic not read			
tunnels (GRE, VXLAN, Geneve)	inner connections forwarded, not fingerprinted; 3 FoxIO captures excluded	not in the scope (no tunnels on h1 – s1 – h2)	a second parser path for the inner headers, the key on the inner endpoints
IPv6 extension headers	such packets forwarded, not fingerprinted	none in the test traffic; not yet written	a parser loop over the extension headers (hop-by-hop, routing, destination options, first fragment)
more than two VLAN tags	forwarded, not fingerprinted	not used on access links	one more parser state per tag
encrypted content: HTTP/2, HTTP/3 requests, TLS 1.3 certificates	no JA4H / JA4X there	impossible without the session keys; FoxIO computes them only from captures that store the keys (16 JA4H and 3 JA4X not compared)	not possible on the wire
Documented limits (the value is missing, never wrong; each one tested)			
a piece before its message's first segment, no SYN seen	that message not reported	without the SYN the offset of an early piece is unknown, so it cannot be placed	not possible without the SYN
a TLS hello after other bytes (SOCKS), split and out of order	not reported	rule (e) covers HTTP directions only; copying after every gap in TLS connections would be load for a rare case	extend (e) to a direction whose first message was not a hello
a TLS 1.2 ServerHello itself split	no JA4X for that flight	the certificate copy starts at a ServerHello that is complete in one segment; a ServerHello is about 100 bytes, so this is rare	start the flight copy when the joined ServerHello comes back

What	Effect	Why it is not done	What doing it would take
two connections in one slot at once	the older loses its remaining state	65,536 hashed slots, detected by the 32-bit check	more slots (one constant) or buckets
lists longer than the capacities	reported “incomplete”	fixed-size fields in P4; measured: 1,238 of 1,238 lists in 42 captures fit	larger fields (cost: metadata and digest size)
Not tested, so not proven			
several Cookie headers in one request	the switch uses the first one’s pairs; FoxIO’s behaviour not checked	no sample has it	a test against FoxIO’s tool
a second ClientHello after a TLS 1.3 HelloRetryRequest	the switch reports the first only (once per direction); FoxIO’s behaviour not checked	no sample has it	a test against FoxIO’s tool
Measurement and evidence			
JA4L on BMv2	the time is stamped when the pipeline takes the packet up; on a busy CPU that is late	a property of the software target, not of the program; the TTL part is exact	a hardware target stamps on arrival
the corpus	FoxIO’s 33 sample captures, the live runs and generated cases, not a large real-world capture	evaluation is the next phase of the project	a real-traffic evaluation (planned)
Deployment (not fingerprinting bugs; see the assumptions table)			
both directions through one switch; one fingerprinting point; BMv2 only	see the assumptions table	properties of where the switch is placed and of the target	placement rules; a hardware port is a separate design

14 Assumptions, and what changes on a real network

What the design assumes, why it holds on h1 – s1 – h2, and what a real network changes. Rows marked **fixed** were found by this review and fixed (tests in Section 5; the problems table). The same table is in `design.pdf` (`docs/assumptions.tex`).

Assumption	Holds here because	On a real network	Status
Where the switch sits			

Assumption	Holds here because	On a real network	Status
both directions of a connection pass through the switch	one path between h1 and h2	with asymmetric routing (ECMP, several uplinks) one switch may see one direction: JA4TS, JA4L, JA4S, JA4SSH and the server-side join need the other	a deployment requirement: place the switch where both directions pass (an access or edge link)
one switch fingerprints a connection	one switch	several P4 switches on a path would each report the same connection	a deployment choice: one fingerprinting point per path, or deduplicate in the controller by endpoints and time
JA4L measures from the switch	the switch is between h1 and h2	JA4L-C is switch↔client, JA4L-S switch↔server: the values depend on where the switch is (FoxIO measures at the server)	correct for an observer in the path; stated with the results
Identifying connections and roles			
endpoints identify a connection	every run used fresh ports	clients and NATs reuse ports	**fixed**: a connection number (ConnId, D21); `test_port_reuse`
the client is the SYN sender	every connection starts with a SYN in the captures	a capture or a switch that starts mid-connection sees no SYN	the ClientHello or request sender is then the client; for SSH on port 22 the port; tested without the SYN (limit cases never wrong)
SSH runs on port 22	sshd listened on 22	SSH on other ports is common	**fixed**: recognised by its banner, as FoxIO's tool does (D23)
frames are untagged	Mininet's links carry no VLANs	access and trunk links carry 802.1Q / 802.1ad tags	**fixed**: up to two tags (D23); more: forwarded, not fingerprinted
two connections never share a slot	a few connections per run	with many connections, two can hash to the same of 65,536 slots	detected by the 32-bit check (never mixed); the older loses its remaining state; slots are one constant
the same addresses are not in use in two VLANs	one network	overlapping address spaces in different VLANs give the same slot	the VLAN id is not part of the key: a one-line change if needed
Load and memory			
split messages are uncommon	most test messages fit in a segment	current Chrome and Firefox ClientHellos carry post-quantum key shares (about 2 KB) and are always split: on browser traffic the controller's join is the common path for JA4	correctness unaffected (the switch still reads every byte); the controller path's throughput is part of the performance evaluation

Assumption	Holds here because	On a real network	Status
the controller's state fits in memory	runs of seconds	runs of days	**fixed** for the reassembler (D24); records are kept until the run ends and written as JSON: a long-running deployment exports and drops them
the machine is idle	tests run alone	BMv2 is software: a busy CPU delays pipeline pick-up, which JA4L measures	JA4L reported as a measurement; the gap test needs an idle machine; hardware stamps on arrival
The target			
the program runs on BMv2 (v1model)	BMv2 is the software target	a hardware target (Tofino) would not compile it as written: parser loops over hundreds of bytes, `advance`, resubmit for every walked packet, 2,048-bit fields, 16 KB windows	the claim is fingerprinting in a P4 data plane on a software target; a hardware port is a separate design
only h1 and h2 send into the switch	Mininet's links	the switch's interfaces (s1-eth1, s1-eth2) live in the VM's own network namespace, so the VM's operating system also sends frames on them (IPv6 neighbour discovery, multicast listener reports, mDNS); the switch takes them in and forwards them like any frame (seen in a live run with the switch's trace log: 30 such frames)	harmless here (nothing the switch fingerprints); on a real switch only the cables' traffic arrives
forwarding is port 1 ↔ port 2	two hosts	a real switch forwards by MAC or IP tables	independent of fingerprinting: the `forward` table can be any forwarding

15 Known limits

- Out of reach without keys: HTTP/2 and HTTP/3 requests, TLS 1.3 certificates. Not implemented: JA4 over QUIC (`q`; readable passively, but v1model cannot decrypt) and DTLS (`d`); tunnels not opened; IPv6 extension headers not walked; more than two VLAN tags not read (forwarded).
- Not tested against FoxIO: several Cookie headers in one request; a second ClientHello after a TLS 1.3 HelloRetryRequest.
- Without the SYN, a piece that arrives before its message's first segment cannot be placed (not reported). A TLS hello after other bytes (SOCKS) split and out of order: the same (rule (e) covers HTTP directions).
- A TLS 1.2 ServerHello that is itself split does not start the certificate copy (no JA4X for that flight).
- Two connections in one slot at the same time: the older loses its remaining state.

- JA4L is stamped when BMv2's pipeline takes the packet up; on a loaded machine that can be late.
- Lists past their capacity are reported incomplete.
- Connection numbers wrap after 2^{32} connections (harmless: a connection with an old number is long over).

16 Questions and answers

Q: What exactly is done in P4, and what is not?

Every byte of every fingerprinted message is read by the P4 parser; the per-connection state, the decision what to copy, the JA4L times and the SSH counts are in the P4 pipeline. Outside P4: sorting, SHA-256, the SSH mode, writing the text, and joining split messages by framing (Section 1, D2–D6).

Q: Is the controller not doing the real work for split messages?

No. It only puts bytes in order by their offsets and cuts at framing boundaries (TLS record and handshake lengths, the empty line, the certificate list's lengths). It then sends the whole message back, and the same P4 parser reads every value. A P4 switch cannot hold packets, so some buffer must exist outside; the reading does not move with it.

Q: How does the switch handle "the server's second packet arriving first"?

From the SYN-ACK the switch knows the server's initial sequence number, so it knows every segment's offset in the server's byte stream. A segment at a non-zero offset in a direction that has not started is copied (rule (c), Fig. 15); when the first segment arrives, the controller has every piece and joins them (Section 3.2). Tested in every order for 2 and 3 pieces, plus reversed 4 and 5 pieces.

Q: Is the server's side (h2) fingerprinted, or only the client's?

Both. h1's ClientHello offers its versions, ciphers, extensions and ALPN: JA4. h2 does not send a list of its own; its ServerHello states its choice (one version, one cipher, the extensions it answers, the ALPN it picked): JA4S. In TLS 1.2 h2 then sends its certificate chain in the clear: one JA4X per certificate (TLS 1.3 encrypts it). h2's SYN-ACK gives JA4TS, its half of the round trip JA4L-S; JA4SSH counts both directions (`c…` h1, `s…` h2); h2's DHCP replies (Offer, Advertise, Reply) get their own JA4D. Only HTTP has no server value: JA4+ fingerprints the request, not the response. Live run, TLS 1.2: `t120300_c030_460f64128655` = TLS 1.2, 3 extensions, no ALPN, cipher `c030` chosen from h1's 28.

Q: h2's messages over several segments, second half first?

Handled the same way as the client's. From the SYN-ACK the switch knows h2's initial sequence number, so every segment of h2 has a known offset. A ServerHello cut in pieces is joined like a ClientHello (rules (a), (b), (c)). For the TLS 1.2 certificate flight: a later segment that arrives before the ServerHello is copied by rule (c) (h2's direction has not started); the segment with the ServerHello starts the flight (rule (d)); later segments are copied by rule (b); the controller cuts the certificates out once there is no gap, and sends each back for the switch's DER walk. Tested: `test_server_flight_reorder`, 19 of 19 orders exact; JA4S also in the 202 split / reorder cases. Limit: a ServerHello that is itself split starts no certificate copy, so JA4X is not reported (never wrong); a TLS 1.2 ServerHello is about 100 bytes, so TCP rarely splits it.

Q: Does the same hold for HTTP and SSH split over packets?

HTTP: yes, for the client's requests (JA4+ has no fingerprint of the server's response). A split request is joined like a hello: 76 of the 202 split / reorder cases are HTTP. A later request on

a keep-alive connection, split and second piece first, needs rule (e): pieces ahead of a gap in an HTTP direction are copied (18 of 18 orders exact), and the connection's JA4H are listed in stream order. SSH: nothing is joined, by definition. JA4SSH counts TCP segments (payload lengths, packets, bare ACKs per window of 200), as FoxIO's tool does; joining would make it differ. Counts do not depend on order, and the mode takes the smallest length on a tie, so reordering inside a window changes nothing.

Q: What happens when a client reuses a port for a new connection?

The SYN with a new sequence number gives the connection a new number (ConnId); every control starts over and the controller opens a new record. An SSH connection's last window is reported on that SYN. A repeated SYN (same sequence number) changes nothing. Before this fix the second connection's JA4 was lost and two SSH sessions were added into one wrong JA4SSH (`test_port_reuse`).

Q: Does it work beyond h1 – s1 – h2?

The fingerprints do not depend on the topology, but some assumptions do: both directions of a connection must pass the switch (asymmetric routing breaks JA4TS, JA4L, JA4S, JA4SSH and the server-side join), one switch should fingerprint a connection, JA4L depends on where the switch is, and the program is written for BMv2, not hardware. Port reuse, VLAN tags, SSH on other ports and bounded controller memory were fixed after this review. Section 14 lists each assumption with its status.

Q: VLAN tags? SSH on another port?

Up to two tags are read (802.1Q, and 802.1ad outside 802.1Q), as FoxIO's tool reads them. SSH is recognised on any port by its "SSH-" banner, as tshark does under FoxIO's tool; bare ACKs before the banner are counted, because FoxIO counts the handshake's ACK (`test_vlan_and_ssh_port`).

Q: And without the SYN?

Then the offsets are learned from the first segment that starts a message; a piece that came before it cannot be placed. The switch reports nothing rather than a wrong value (8 limit cases, tested).

Q: Why are packets resubmitted? Does that change them?

The parser skips bytes with `advance`, which removes them from the emitted packet. Resubmitting runs the original packet again; the second pass parses only Ethernet, so all bytes are forwarded unchanged. Checked on every run, frame by frame (9,013 of 9,013).

Q: Why a hash of the endpoints and not a table entry per connection?

The data plane cannot add table entries, and waiting for the controller would lose the first packets (and JA4L's SYN time). Hashed register slots exist from the first packet; a 32-bit check value detects another connection in the same slot (D8), and a connection number tells a new connection on reused ports from the old one (D21).

Q: Why does the controller sort and hash instead of the switch?

v1model has no SHA-256 and no loops in the pipeline; the fingerprint is fixed by the values the switch read, so sorting and hashing them elsewhere gives the same value. A sorting network in P4 would only be needed on hardware to emit sorted lists (D4).

Q: Why 128 ciphers, 64 extensions, 512 bytes of names …?

Chosen generously, then measured: all 1,238 lists of 42 captures fit (Section 12). Past a capacity the fingerprint is reported incomplete, never truncated (tested).

Q: Why does a digest carry a serial number?

BMv2 drops a digest identical to one not yet acknowledged; identical DHCP retransmissions, repeated HTTP requests and identical SSH windows were lost. 32 bits is far more than the number of unacknowledged digests can reach; wrapping is harmless because the value is never read (D10).

Q: Why is JA4L not scored exact?

It is a time. BMv2 stamps a packet when its pipeline takes it up, which a software switch delays (median 0.33 ms in replay). The TTL part is scored; the latency is reported as a distribution; the arithmetic is tested with known 1 s / 0.5 s gaps.

Q: How do you know the fingerprints are right?

FoxIO's own tools compute the reference on the same frames: 573 exact, 5 differences explained by hand (the reference cannot decode those streams), 0 wrong, 0 missing, 0 extra; the live Mininet run with real programs adds every fingerprint again: 49 of 49 exact, 196 of 196 frames forwarded unchanged.

Q: What are the 5 EXPLAINED rows?

One SOCKS4 ServerHello on port 9901 whose ClientHello is missing from the capture, so FoxIO's tool decodes no TLS there (decoded by hand: equal to the switch). Four SSH streams that are malformed or SSH version 1, where tshark refuses a few segments as SSH; the switch counts every port-22 payload segment.

Q: Is fingerprinting complete in every respect?

No, and the project does not claim it. What the evidence supports: for clear-text TCP and DHCP traffic, every JA4+ fingerprint computable without keys matches FoxIO's tools, for messages whole, split, reordered, duplicated or overlapping, from both sides (573 exact on FoxIO's samples, 47 of 47 in the final live run, 27 tests). Not done: JA4 over QUIC and DTLS, tunnels, IPv6 extension headers; two cases are not tested (several Cookie headers, a HelloRetryRequest's second ClientHello). Section 13 gives each item with the reason it is not done and what doing it would take.

Q: What is not covered?

Section 15, known limits: encrypted content (HTTP/2, HTTP/3, TLS 1.3 certificates), QUIC and DTLS, tunnels, IPv6 extension headers, more than two VLAN tags; and the deployment assumptions of Section 14.

Q: Can a higher-level program drive the switch?

Yes: through P4Runtime, as the controller does (write `forward`, enable digests, send packet-out). The controller's Python class is the API; another P4Runtime client can replace it.

Q: Where does each result file come from?

`results/compare_all.json`: `make compare`; `results/live/`: `make live` (which also writes `results_live.tex`); `results/measure/`: `tools/measure_field_counts.py`; the PDFs' tables: `docs/results_*.tex`, written from those files.

Q: How is this document kept current?

Both PDFs include the same problems table (`docs/problems.tex`) and results tables (`docs/results_*.tex`). Every fix adds a row to the problems table; every design change updates the matching section here. `make docs` builds both.